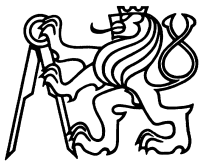
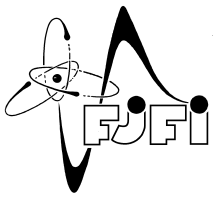


Czech Technical University in Prague
Faculty of Nuclear Sciences and Physical Engineering



High performance computing methods for numerical solution of problems with phase transitions

Metody vysoce výkonného počítání pro numerické řešení úloh s fázovými přechody

Bachelor's Degree Project

Author: **Kryštof Jakůbek**

Supervisor: **Ing. Pavel Strachota, Ph.D.**

Academic year: 2023/2024

ZADÁNÍ BAKALÁŘSKÉ PRÁCE

I. OSOBNÍ A STUDIJNÍ ÚDAJE

Příjmení: **Jakůbek** Jméno: **Kryštof** Osobní číslo: **509234**

Fakulta/ústav: **Fakulta jaderná a fyzikálně inženýrská**

Zadávající katedra/ústav: **Katedra matematiky**

Studijní program: **Matematické inženýrství**

Specializace: **Matematická informatika**

II. ÚDAJE K BAKALÁŘSKÉ PRÁCI

Název bakalářské práce:

Metody vysoce výkonného počítání pro numerické řešení úloh s fázovými přechody

Název bakalářské práce anglicky:

High performance computing methods for numerical solution of problems with phase transitions

Pokyny pro vypracování:

1. Prostudujte modely dynamiky kontinua popisující fyzikální procesy s fázovými přechody. Zaměřte se na popis tuhnutí a tání, proudění a silové interakce mezi pevnou a kapalnou fází.
2. Seznamte se s modelováním fázových přechodů metodou fázového pole.
3. Seznamte se základy numerických metod pro řešení matematických modelů výše zmíněných procesů.
4. Diskutujte možnosti paralelního zpracování výsledných algoritmů na různých typech architektur pro vysoce výkonné počítání (mnohajádrové systémy, distribuované systémy, GPU akcelerátory).
5. Implementujte numerický řešič typové úlohy tuhnutí a tání materiálu a pokuste se o jeho paralelizaci. Použijte vhodnou numerickou metodu, např. metodu konečných diferencí nebo metodu konečných objemů.
6. Prezentujte a komentujte výsledky numerických simulací.

Seznam doporučené literatury:

[1] N. Provatas, K. Elder, Phase-Field Methods in Materials Science and Engineering. WILEY-VCH Verlag GmbH & Co. KGaA, 2010.
[2] Y. Wang et al., Simulation of Microstructure Evolution in Mg Alloys by Phase-Field Methods: A Review. Crystals 12, 2022, 1305.
[3] P. Strachota, A. Wodecki, M. Beneš, Focusing the latent heat release in 3D phase field simulations of dendritic crystal growth. Modelling Simul. Mater. Sci. Eng. 29, 2021, 065009.
[4] I. Steinbach, Phase-field models in materials science. Modelling Simul. Mater. Sci. Eng. 17, 2009, 073001.
[5] T. Takaki, S. Sakane, R. Suziki, High-performance GPU computing of phase-field lattice
Boltzmann simulations for dendrite growth with natural convection. IOP Conf. Series: Materials Science and Engineering 1281, 2023, 012056.
[6] F. Moukalled, L. Mangani, M. Darwish et al., The finite volume methods in computational fluid dynamics. Springer, 2016.
[7] J. Palán, Modely fázového pole v materiálových vědách a jejich numerické řešení. Diplomová práce, FJFI ČVUT v Praze, 2023.

Jméno a pracoviště vedoucí(ho) bakalářské práce:

Ing. Pavel Strachota, Ph.D. katedra matematiky FJFI

Jméno a pracoviště druhé(ho) vedoucí(ho) nebo konzultanta(ky) bakalářské práce:

Datum zadání bakalářské práce: **24.10.2023** Termín odevzdání bakalářské práce: **05.08.2024**

Platnost zadání bakalářské práce: **30.09.2025**

Ing. Pavel Strachota, Ph.D.	prof. Ing. Zuzana Masáková, Ph.D.	doc. Ing. Václav Čuba, Ph.D.
podpis vedoucí(ho) práce	podpis vedoucí(ho) ústavu/katedry	podpis děkana(ky)

III. PŘEVZETÍ ZADÁNÍ

Student bere na vědomí, že je povinen vypracovat bakalářskou práci samostatně, bez cizí pomoci, s výjimkou poskytnutých konzultací. Seznam použité literatury, jiných pramenů a jmen konzultantů je třeba uvést v bakalářské práci.

28. 11. 2023 — Datum převzetí zadání

Podpis studenta

Acknowledgment:
I would like to thank my supervisor Ing. Pavel Strachota, Ph.D. for his invaluable insight, critique and seemingly boundless patience.

Author's declaration:
I declare that this Bachelor's Degree Project is entirely my own work and I have listed all the used sources in the bibliography.

Prague, January 6, 2025 Kryštof Jakůbek

Název práce:

Metody vysoce výkonného počítání pro numerické řešení úloh s fázovými přechody

Autor: Kryštof Jakůbek

Specializace: Matematické inženýrství

Specializace: Matematická informatika

Druh práce: Bakalářská práce

Vedoucí práce: Ing. Pavel Strachota, Ph.D., Fakulta jaderná a fyzikálně inženýrská, Katedra matematiky

Abstrakt: Tato práce se zabývá paralelní implementací na GPU numerických schémat pro řešení dvourozměrného modelu fázového pole, popisujícího růst krystalů v podchlazené kapalné fázi. Nejprve je představen model fázového pole. Metoda konečných objemů je využita k odvození semi-diskrétního schématu pro přípustné sítě. Toto schéma je numericky integrováno pomocí explicitních metod vyššího řádu. Poté je odvozeno semiimplicitní schéma časové integrace pomocí Crank-Nicolsonovy metody, které je řešeno pomocí metody konjugovaných gradientů. Dva přístupy ke snížení chyby způsobené použitou metodou štěpení operátorů jsou prezentovány a později porovnány. Je důkladně představeno programování pomocí CUDA a je uvedeno několik simulací využitých optimalizovaných algoritmů. V testu je ukázána efektivita jednoho z popsaných algoritmů. Nakonec jsou porovnány výsledky simulací navržených schémat časové integrace a je ukázána schoda s dřívejšími výsledky.

Klíčová slova: CUDA, krystalizace, metoda fázového pole, metoda konečných objemů, paralélní výpočty

Title:

High performance computing methods for numerical solution of problems with phase transitions

Author: Kryštof Jakůbek

Abstract: This work is concerned with GPU parallel implementation of numerical schemes of the two dimensional phase field model, describing crystal growth in undercooled media. Firstly, the phase field model is introduced and the finite volume method is utilized to derive a semi-discrete scheme for admissible meshes. This scheme is numerically integrated using higher order explicit methods. Then, a semi-implicit time integration scheme is derived using the Crank-Nicolson method and solved using the conjugate gradient method. Two approaches to reduce the error introduced by the operator splitting method are presented and later compared. Programming with CUDA is thoroughly introduced and several optimized algorithms required by the simulation implementation are explained. The efficiency of one of the described algorithms is shown in a benchmark. Finally, simulation results of the proposed time integration schemes are compared and good agreement with previous results is shown.

Key words: CUDA, crystal growth, finite volume method, parallel processing, phase field model

Contents

Summary of Notation

Ω	Computational domain
$\mathcal{T}$	Time interval
Φ	Phase field
T	Temperature field
$a, b, \alpha, \beta, \xi, L, T^*$	Model parameters
$\mathcal{M}$	Set of finite volume cells over the domain Ω
C, D	Grid cells
∂C	Boundary of grid cell C
$\sigma = C\|D$	Cell face between cells C and D
$m(C)$, $\tilde{m}(\sigma)$	Lebesgue measure of cell C and cell face σ
$\rho(x, y)$	Distance between points x and y
$\rho(x, \sigma) = \inf_{y \in \sigma} \rho(x, y)$	Distance between point x and cell face σ
$\boldsymbol{x}_C$	Position in the centroid of cell C
E, W, N, S	Cells to the East, West, North and South relative to cell C, respectively
$\boldsymbol{e}_1, \boldsymbol{e}_2, \boldsymbol{e}_3$	Ortonormal basis vectors of $\mathbb{R}^3$
$\Gamma(t)$	Approximate shape of the phase interface

Introduction

Solidification forms a crucial step in many practical applications including metallurgy, semiconductor manufacturing and optics. The microscopic crystal structure of the resulting material can have profound impact on its mechanical, electric and magnetic properties. Because of this, simulating the crystal structures ahead of time can be of interest. Goal of this work is to derive and efficiently implement a numerical scheme for solving the phase field problem, governing solidification of undercooled media and dendritic crystal growth. While other methods have been proposed and implemented with excellent scaling properties on many core setups [SB16], [Pal23], they are limited to the CPU. Thus, to achieve fast simulation running times, access to specialized high performance computing cluster is required. We present a numerical scheme and its implementation capable of achieving good runtime on consumer hardware. For this purpose the implementation has been devised to run efficiently on the GPU, which provides excellent compute capabilities for massively parallel workloads encountered in the simulation.

This work is divided into five chapters. Chapter 1 provides an overview of the models of solidification in undercooled media, which will be utilized in the rest of the text. For this Stefan and phase field problems are introduced. Models of fluid flow incorporating the interaction between fluid and solid particles are briefly reported on.

The Chapter 2 is focused on the finite volume method. The general finite volume framework is introduced and then utilized to derive the discretization of the gradient and Laplacian operators. Then, common boundary conditions are described within the finite volume framework. Lastly, the phase field model described in Chapter 1 is discretized into the so-called semi-discrete scheme.

The Chapter 3, is concerned with the derivation of time integration schemes for the semi-discrete scheme from Chapter 2. Two categories of schemes are derived. First category are the explicit schemes, which solve the semi-discrete scheme by direct calculation. Second category are the semi-implicit schemes, which reformulate the problem into a system of linear equations and solve it using an appropriate matrix solver. The error caused by approximations done during the time integration scheme derivation is quantified and two approaches for reducing this error are presented.

The Chapter 4 thoroughly introduces programming on the GPU utilizing the CUDA programming model. It details several optimized algorithms used during the implementation of schemes derived in Chapter 3. A special effort was put on the explanation in hopes that even an unfamiliar reader may understand the algorithms presented. It showcases the efficiency of one of the presented algorithms by comparing it with the CUDA Thrust library.

The Chapter 5 presents and discusses the results obtained by running the developed simulation. Effect of various model parameters as well as boundary conditions is shown. Then the schemes proposed in Chapter 3 are compared and the impact of the described error reduction approaches presented is evaluated.

Chapter 1

Mathematical modeling of phase transitions

Solidification is a phase transition of the first order occurring mainly as a result of temperature change. However, solidification of undercooled liquid only occurs upon a slight energy disturbance, typically caused by impurities in the liquid around which an initial nucleus of solid phase is formed. This initial nucleus then interacts with its surroundings, causing it to solidify as well, resulting in crystal growth. The changing shape of the solid region combined with different conditions in each of the material phases poses a unique modeling challenge. For this, two approaches of modeling solidification were developed: the first ones are the sharp interface models, which explicitly track the position of the interface between the solid and liquid phases [Sch96]. The second ones are the phase field models, sometimes also called smooth interface models, which utilize a scalar field to represent the phase of the material in the specific point in time and space. A rapid but smooth transition between the phases is assumed [Raa98].

The phase field approach has been successfully used to simulate, amongst others dendritic growth in pure supercooled melts [WMS93], in binary alloys [WBM92] and in interaction with fluid flows [BDP22].

This text will focus on solidification of pure undercooled substances in two dimensions utilizing the phase field model, still, a general introduction is given. We first introduce the Stefan problem with surface tension, which is a sharp interface model, then present its corresponding phase field formulation, which will be studied further. Finally, we report on the models of hydro-mechanical phenomena.

1.1 The Stefan problem

Consider domain $\Omega \subset \mathbb{R}^3$ alongside time interval $\mathcal{T} = [t_{\text{begin}}, t_{\text{end}}]$. At each $t \in \mathcal{T}$ the domain Ω is divided into a solid subdomain $\Omega_s(t)$ and a liquid subdomain $\Omega_l(t) = \Omega \setminus \Omega_s(t)$, separated by a phase interface $\Gamma(t) = \partial\Omega_s(t) \cap \partial\Omega_l(t)$. The Stefan problem [Ben01b] is given as

$$\rho c \frac{\partial T}{\partial t} = \nabla(\lambda \nabla T) \qquad \text{in } \mathcal{T} \times \Omega_s(t) \text{ and } \mathcal{T} \times \Omega_l(t), \tag{1.1}$$

$$b_c(u)|_{\partial\Omega} = 0 \qquad \text{on } \mathcal{T} \times \Omega, \tag{1.2}$$

$$T|_{t=0} = T_{ini} \qquad \text{in } \Omega, \tag{1.3}$$

$$\lambda \frac{\partial T}{\partial \boldsymbol{n}_\Gamma}\bigg|_s - \lambda \frac{\partial T}{\partial \boldsymbol{n}_\Gamma}\bigg|_l = L\nu_\Gamma \qquad \text{on } \Gamma(t), \tag{1.4}$$

$$T - T^* = -\frac{\sigma}{\Delta s}\kappa_\Gamma - \alpha\frac{\sigma}{\Delta s}\nu_\Gamma \qquad \text{on } \Gamma(t), \tag{1.5}$$

$$\Omega_s(0) = \Omega_{s,ini}, \tag{1.6}$$

where the $T : \mathcal{T} \times \Omega \to \mathbb{R}$ is temperature field. The model parameters $\rho, c, \lambda, L, u*, \sigma, \Delta s, \alpha \in \mathbb{R}$ alongside $\boldsymbol{n}_\Gamma \in \mathbb{R}^3, \nu_\Gamma, \kappa_\Gamma \in \mathbb{R}$ are described in Table 1.1. In the Stefan problem, (1.1) is the heat equation, evaluated separately in the liquid and solid subdomains, (1.2) is the Gibbs-Thomson relation and (1.4) is the Stefan condition, describing heat flow discontinuity at the phase interface Γ. $\boldsymbol{n}_\Gamma$ is the normal vector to $\Gamma(t)$, b_c is the boundary condition operator which can be taken as

$$b_c(T) = T - T^* \qquad \text{for the Dirichlet boundary condition or}$$
$$b_c(T) = (\lambda\nabla T - \boldsymbol{g}) \cdot \boldsymbol{n}_{\partial\Omega} \qquad \text{for the Neumann boundary condition,}$$

where vector $\boldsymbol{g}$ specifies heat flux through $\partial\Omega$ and $\boldsymbol{n}_{\partial\Omega}$ is the normal to $\partial\Omega$ in the given point.

ρ	material density	T^*	material melting point
λ	heat conductivity	Δs	difference in entropy per unit volume
L	latent heat of fusion per unit volume	α	coefficient of attachment kinetics
ν_Γ	normal velocity of $\Gamma(t)$	σ	surface tension
$\boldsymbol{n}_\Gamma$	normal vector to $\Gamma(t)$	c	material specific heat capacity
κ_Γ	mean curvature of $\Gamma(t)$		

Table 1.1: Parameters of the the Stefan problem

1.2 Phase field problem

We transition from the Stefan problem (1.1) - (1.6) to its phase field formulation. We consider the phase field $\Phi : \mathcal{T} \times \Omega \to [0, 1]$, which represents phase of the material in the given point in time $t \in \mathcal{T}$ and space $\boldsymbol{x} \in \Omega$. Points where $\Phi(t, \boldsymbol{x}) < 1/2$ are considered liquid and points where $\Phi(t, \boldsymbol{x}) \geq 1/2$ are consider solid. Specifically the phase interface $\Gamma(t)$ is defined as

$$\Gamma(t) = \left\{ \boldsymbol{x} \in \Omega, \Phi(t, \boldsymbol{x}) = \frac{1}{2} \right\},$$

which lets us to represent $\boldsymbol{n}_\Gamma$, ν_Γ and κ_Γ as

$$\boldsymbol{n}_\Gamma = -\frac{\nabla\Phi}{|\nabla\Phi|}, \qquad \nu_\Gamma = \frac{1}{|\nabla\Phi|}\frac{\partial\Phi}{\partial t}, \qquad \kappa_\Gamma = \nabla \cdot \boldsymbol{n}_\Gamma. \tag{1.7}$$

The phase field problem given by [Ben01b] reads

$$\frac{\partial T}{\partial t} = \nabla^2 T + L\frac{\partial \Phi}{\partial t} \qquad \text{in } \mathcal{T}\times\Omega, \tag{1.8}$$

$$\xi^2\alpha\frac{\partial \Phi}{\partial t} = \xi^2\nabla^2\Phi + f(\Phi, T, \nabla\Phi) \qquad \text{in } \mathcal{T}\times\Omega, \tag{1.9}$$

$$T|_{t=0} = T_{ini} \qquad \text{in } \Omega, \tag{1.10}$$

$$\Phi|_{t=0} = \Phi_{ini} \qquad \text{in } \Omega, \tag{1.11}$$

$$T|_{\partial\Omega} = T_{\partial\Omega} \text{ or } \nabla T\cdot\boldsymbol{n} = 0 \qquad \text{in } \mathcal{T}\times\partial\Omega, \tag{1.12}$$

$$\Phi|_{\partial\Omega} = \Phi_{\partial\Omega} \text{ or } \nabla\Phi\cdot\boldsymbol{n} = 0 \qquad \text{in } \mathcal{T}\times\partial\Omega, \tag{1.13}$$

where L, α, β are dimensionless forms of the original parameters of the Stefan problem (1.1) - (1.6). They are calculated as

$$L = \frac{L_{\text{real}}}{\rho c\Delta T_{ini}}, \qquad \alpha = \alpha_{\text{real}}\frac{\lambda}{\rho c}, \qquad \beta = \frac{\Delta s}{\sigma}L_0\Delta T_{ini},$$

where we used L_{real}, α_{real} to refer to the original parameters in Section 1.1. $\Delta T_{ini} = T_{ini} - T^*$ is the uniform initial undercooling, t_0 is the time scale and $L_0 = \sqrt{\frac{\lambda}{\rho c}t_0}$ is the length scale.

In (1.9), $f(\Phi, T, \nabla\Phi)$ is the so-called reaction term. Multiple forms of the reaction term have been proposed with distinct physical and numerical properties [Cag89], [Kob93], [SWB21]. We will use the form given in [Ben01b], which is written as

$$f(\Phi, T, \nabla\Phi) = af_0(\Phi) - b\xi^2\beta|\nabla\Phi|(T - T^*), \tag{1.14}$$

$$f_0(\Phi) = \Phi(1-\Phi)(\Phi - \frac{1}{2}). \tag{1.15}$$

Lastly, the parameter $\xi > 0$ controls the width of the phase transition. It has been shown that given certain conditions, the solution to the phase field problem converges to the solution of the original Stefan problem with $\xi \to 0_+$ [Ben97].

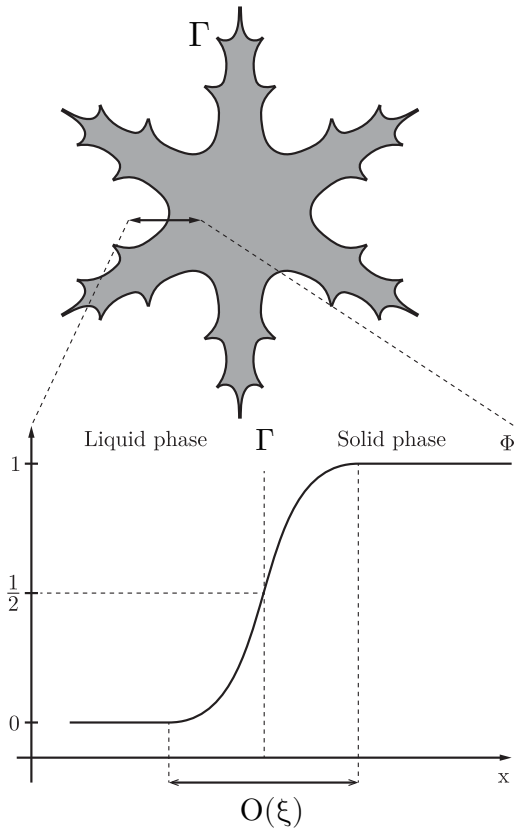


Figure 1.1: Phase field Φ, phase interface Γ and the shape of phase transition, whose width is proportional to ξ

The characteristic shape of the phase transition is depicted in Figure 1.1. The shape is the result of $f_0(\Phi) = \frac{\mathrm{d}\omega_0(\Phi)}{\mathrm{d}\Phi}$, where ω_0 is the bulk free-energy density, defined as $\omega_0(\Phi) = a\Phi^2(1-\Phi)^2/4$ [Cag89]. The two local minima of ω_0 in values 0 and 1, corresponding to the liquid phase and solid phase respectively, are shown in Figure 1.2.

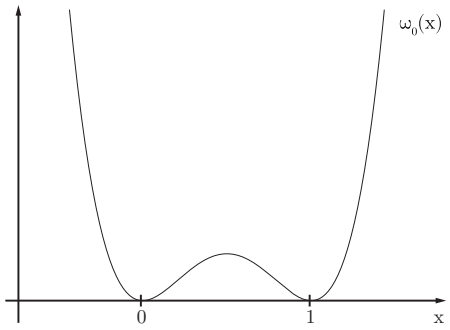


Figure 1.2: the double well potential ω_0

1.2.1 Anisotropy

The phase field model described in Section 1.2 is not sufficient to generate the complex crystal structures found in nature. One common characteristic of such structures is anisotropy, that is, stronger tendency of the crystal to grow in certain directions compared to others. Models of anisotropy utilizing the concept of surface motion and Finsler geometry have been proposed and successfully implemented [Ben03], [BP96], [Str12]. However, for simplicity, we will use the two dimensional model given in [Ben01a]

$$\xi^2\alpha\frac{\partial\Phi}{\partial t} = g(\theta)\xi^2\nabla^2\Phi + g(\theta)af_0(\Phi) - b\xi^2\beta|\nabla\Phi|(T-T^*), \tag{1.16}$$

$$g(\theta) := 1 - S\cos(m(\theta_0-\theta)), \tag{1.17}$$

where $\theta \in \mathbb{R}$ denotes the angle between the phase interface normal and the basis vector $\boldsymbol{e}_1$, $S \in [0,1)$ is strength of m-fold anisotropy and $\theta_0 \in \mathbb{R}$ is principal crystallographic orientation [Ben01b]. Note that θ can be computed using

$$\theta = \operatorname{atan2}(\frac{\partial\Phi}{\partial y}, \frac{\partial\Phi}{\partial x}). \tag{1.18}$$

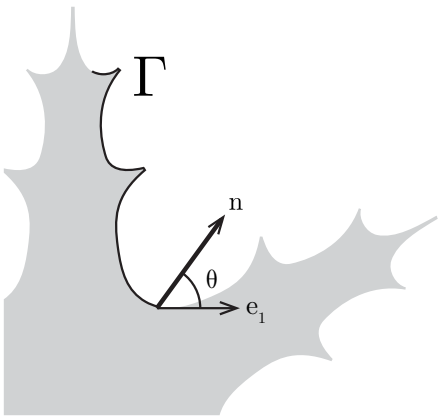


Figure 1.3: Angle θ between the phase interface normal $\boldsymbol{n}$ and basis vector $\boldsymbol{e}_1$

To simplify notation in the upcoming sections we will rewrite the phase field problem equations (1.16), (1.8) using nonlinear functions $k1, k2, k3$

$$\frac{\partial\Phi}{\partial t} = \underbrace{\frac{g(\theta)}{\alpha}}_{k_1(\nabla\Phi)}\nabla^2\Phi + \underbrace{\frac{g(\theta)af_0(\Phi)}{\xi^2\alpha}}_{k_0(\nabla\Phi)} + \underbrace{\frac{b\beta|\nabla\Phi|}{\alpha}}_{k_2(\nabla\Phi)}(T^*-T),$$

as

$$\frac{\partial T}{\partial t} = \nabla^2 T + L\frac{\partial\Phi}{\partial t}, \tag{1.19}$$

$$\frac{\partial\Phi}{\partial t} = k_1(\nabla\Phi)\nabla^2\Phi + k_0(\Phi,\nabla\Phi) + k_2(\nabla\Phi)(T^*-T). \tag{1.20}$$

This formulation will be used in the remainder of the text.

1.3 Hydro-mechanical phenomena

In this section, we report on models describing fluid-structure interaction (FSI). This exploration was motivated by modeling of freezing water in porous media, where under certain conditions, mechanical effects can be very significant. First, we summarize various approaches to FSI and describe in detail, the method presented in [Red+21]. Second, we focus on poromechanical interactions of freezing water and describe the model given in [ŽBI23], [ŽB18].

1.3.1 Fluid structure interaction

Two broad categories for modeling FSI are used. One category of approaches are the body-fitted mesh methods, such as the arbitrary Lagrange-Euler method [JT96]. The simulation grid is dynamically adjusted to the movement of the solids, while ensuring the grid nodes are located at the solid-liquid interface. For significant changes to the grid, for example caused by the solid's rotation, the grid needs to be remeshed, which incurs high computational cost and errors resulting from interpolation of fields into the new grid [Bas+17], [BDP22]. Another option is the immersed boundary method (IBM) [Pes72], which solves the Navier-Stokes equations, while accounting for interactions with one dimensional fibers described in Lagrangian coordinates.

The second category is based entirely on Eulerian description of both liquids and solids. One such approach is to model the solids as extremely high viscosity Newtonian fluids and solve the standard Navier-Stokes equations [AMW00]. This approach is conceptually simple, but suffers from poor numerical performance and does not represent the constitutive behaviour of solids [Sal05]. Next, [Ton+01], [LBK01] investigate the effects of convection on dendritic crystal growth in the phase field model. However, the growing particles are kept stationary, thus the full extent of FSI is not modeled. Another set of methods are the distributed Lagrange multiplier methods (DLM) on a fictious domain, which enforced rigid body motion for the solid via a field of Lagrange multipliers. Examples of this method include [Gal+14], [Red+21], which show promising results in modeling the fluid-structure, structure-fluid and structure-structure interactions for large amounts of separate particles of various geometries. A rough description of the model presented in [Red+21] follows.

The DLM method given in [Red+21] models the interaction between a set of particles and the surrounding incompressible fluid. Both fluid-structure and structure-structure interactions are considered. Each particle is represented using a phase field defined similar to the model given in Section 1.2. In total, N phase fields are tracked to represent N bodies.

Fluid-structure interaction is achieved by transport of the phase fields by the fluid flow. The Navier-Stokes equations are solved in the whole domain, without considering the solid particles, represented by the so-called ghost fluid. Then using the resulting velocity field, a barycenter velocity and angular velocity are calculated for each particle. This motion is subsequently re-projected into the velocity field in the region covered by said particle. This results in a near constitutive behavior of the solid particles. Still, some error is committed, thus the Allen-Cahn equation [AC79], governing the shape of the phase field, is solved every couple of time steps. This restores the phase interface into the expected characteristic shape.

Structure-structure interaction is detected by utilizing the smooth transition between phases in the phase-field model. If the value of phase fields for both of the particles is higher than a certain amount and the particles are moving in opposite directions, the particles are considered to be interacting. The interaction is resolved using the particle distance and the calculated barycenter velocities. This procedure is repeated for all contact points in a single time step.

This model successfully resolves interactions of many spherical particles immersed in a fluid, interaction of spherical particles and static complex mesh as well as simultaneous FSI and grain growth.

1.3.2 Poromechanical interaction

Variety of approaches modeling water freezing is porous media have been developed. For some early macroscopic models see [Gil80], [Fow89]. [ZM13],[Zhe+17] used theory of porous media to derive a fully integrated thermo-hydro-mechanical model able to describe liquid transport within pores, latent heat effect and the deformation of the solid. The remainder of this section will be devoted to describing the models given in [ŽBI23] and [ŽB18].

The model given in [ŽBI23] describes water freezing on a microscopic (pore-scale) level. The geometry of the pores created between arbitrary shaped particles is used directly without any spatial averaging. Because of this, it may provide more accurate results on the micro scale.

In any given point of the domain there may be water, ice or solid particle. Each of the materials is equipped with momentum and energy conservation equations. Ice and solid particle materials are assumed to be linearly elastic, while water is assumed to be a slightly compressible Newtonian fluid. Next, the materials are linked with each other through equations of heat transfer through their shared boundary. Transfer through the water-ice boundary is modeled using the Stefan condition 1.4.

The phase field model with anisotropy similar to the model given in Section 1.2 is employed to describe the motion of the water-ice boundary. Quantities on the smooth water-ice interface are assumed to depend linearly on the value of the phase field in the given point. Expansion of the ice phase is captured with slight modification of the water and ice stress tensor. Finally, boundary conditions for heat fluxes and stress tensors of the materials are given. The phase field is assumed to be liquid at the water/ice-particle boundary, helping the model avoid singularities related to the shared water-ice-particle boundary, while also more closely modeling experimentally observed phenomena.

Via computational studies the resulting model is shown to be able to describe various structural dynamics of the solid phase within complex solid particle geometries. The ice material is shown to spread and fill in the pore space as well as be transported by the created fluid flow.

Earlier study by the same authors [ŽB18] has shown the viability of a similar model to describe displacement of the material as a result of ice phase expansion.

Chapter 2

Finite volume method

Since analytical solution of the phase field problem given in Section 1.2 is not known, we attempt to obtain a numerical solution. The first step towards numerical solution of a parabolic PDEs is spatial discretization, for which we choose the finite volume method (FVM).

The FVM is a discretization method broadly used in simulations of fluid dynamics, heat and material transfer problems [Bla15]. The FVM divides the computational domain into a finite number of regions of polygonal or polyhedral shape called *elements* or *cells*. Then the studied partial differential equation are integrated over each of the elements, yielding a discretized system of algebraic equations, which are then solved to compute the values of the dependent variables [MMD16].

Because fluxes are shared between the neighbouring elements, that is a flux entering an element is equal the flux leaving its neighbour, the FVM is locally conservative. This makes it a good fit for discretizing conservation laws [MMD16].

Another advantage of the FVM is its versatility with regards to the structure and geometry of the cells used, the so called mesh. Meshes can be broadly classified into *structured* and *unstructured*. Cells in a structured mesh are specified in well defined order, for example in a grid of uniformly sized rectangular or triangular cells. Cells in unstructured meshes are in generally arbitrary order, only defined by connectivity. This allows for simulation of problems on complex geometries.

We introduce the formalism of finite volume meshes, derive the spatial discretization of Laplacian and gradient operators on a simple structured mesh, implement boundary conditions using ghost cells, and finally formulate a semi-discrete scheme of the phase field model.

2.1 General mesh

Let Ω denote an open bounded polygonal or polyhedral subset of $\mathbb{R}^d$ where $d \in 2, 3$. Let m resp. $\tilde{m}$ be the d-dimensional resp. $(n-1)$-dimensional Lebesgue measure. Where the dimensionality is clear from the context we will use m to also denote $\tilde{m}$. Further let $\rho(\boldsymbol{x}, \boldsymbol{y})$ be an L_2 distance between points $\boldsymbol{x}, \boldsymbol{y} \in \mathbb{R}^n$.

The domain Ω is covered by a finite volume mesh $\mathcal{M} \in 2^\Omega$ of polygonal cells aka control volumes. Let ε be a set of $(d-1)$-dimensional faces aka edges of non-zero measure.

Definition The mesh $\mathcal{M}$ is considered *admissible* if the following conditions are satisfied [SB18]

1. $\overline{\bigcup_{C\in\mathcal{M}} C} = \overline{\Omega}$.
   The mesh covers the entire domain.
2. $(\forall C \in \mathcal{M})(\exists \varepsilon_C \subset \varepsilon)(\partial C = \bigcup_{\sigma\in\varepsilon_C} \overline{\sigma})$.
   The boundary of a cell is an union of several edges.
3. $\varepsilon = \bigcup_{C\in\mathcal{M}} \mathcal{M}_C$.
   All edges correspond to some cell.
4. $(\forall C, D \in \mathcal{M})(C \neq D \implies (\tilde{m}(\partial C \cap \partial D) = 0 \vee (\exists \sigma \in \varepsilon)(\partial C \cap \partial D = \sigma)))$.
   Two cells either share exactly one edge or no edges at all. We will denote the shared edge between cells C and D as $C|D$.
5. $(\exists \mathcal{P} \subset \Omega)((\forall C \in \mathcal{M})(\exists_1 \boldsymbol{x}_C \in \overline{C}) \wedge (\forall \boldsymbol{x} \in \mathcal{P})(\exists_1 C \in \mathcal{M})(\boldsymbol{x} \in \overline{C}))$.
   There is a set of cell centroids $\mathcal{P}$, such that there is precisely one for each cell.
6. $(\forall C, D \in \mathcal{M})(C \neq D \implies (\boldsymbol{x}_C \neq \boldsymbol{x}_D \vee \overline{\boldsymbol{x}_C\boldsymbol{x}_D} \perp C|D))$.
   The connecting line $\overline{\boldsymbol{x}_C\boldsymbol{x}_D}$ between centroids of two adjacent cells runs perpendicularly to the corresponding cell face.

In the rest of the text we will only consider admissible meshes. In addition to the symbols introduced in the above definition, let us for a general function $\Psi : \Omega \to \mathbb{R}$ define the following.

- Let $\mathcal{M}_{\text{ext}} := \{C \in \mathcal{M} | \partial C \cap \partial\Omega \neq \emptyset\}$ be a set of boundary cells and $\mathcal{M}_{\text{int}} := \mathcal{M} \setminus \mathcal{M}_{\text{ext}}$ be a set of interior cells.
- Let $\varepsilon_{\text{ext}} := \{\sigma \in \varepsilon | \sigma \cap \partial\Omega \neq \emptyset\}$ be a set of boundary faces and $\varepsilon_{\text{int}} := \mathcal{M} \setminus \varepsilon_{\text{ext}}$ be a set of interior faces.
- Let $\Psi_d : \mathcal{P} \to \mathbb{R}$ be a mesh function approximating values of Ψ at cell centroids $\boldsymbol{x} \in \mathcal{P} \subset \Omega$
- Let Ψ_C denote the value of the mesh function Ψ_d at the centroid of cell $C \in \mathcal{M}$, that is $\Psi_C = \Psi_d(\boldsymbol{x}_C)$.
- Let Ψ_σ mark the value of Ψ_d at the cell face $\sigma \in \varepsilon$ obtained through interpolation. The interpolation used is dependent on the specific mesh and will be specified for each case individually.

Consider a PDE for unknown function $\Psi : \mathcal{T} \times \Omega \to \mathbb{R}$. By the notation $F(\Psi) \approx F_d(\Psi)$ we denote the substitution of the term $F(\Psi)$ in the original ODE by certain term $F_d(\Psi_d)$, where Ψ_d is a mesh function

$$\Psi_d : \mathcal{T} \times \mathcal{P} \to \mathbb{R}.$$

By substituting all terms in the original ODE, the so called *semi discrete scheme* is obtained. We will not attempt to quantify the error caused by the substitutions used. Nevertheless, we will state the order of approximation for a given substitution when one is known.

2.2 Structured mesh

We define a structured mesh that will be used in the majority of the text. Let $\Omega = (0, L_{0x}) \times (0, L_{0y}) \subset \mathbb{R}^2$ where L_{0x} resp. L_{0y} is the size of the domain along the $\boldsymbol{e}_1$ resp. $\boldsymbol{e}_2$. We divide this domain into n_x, $n_y \in \mathbb{N}$ equally sized rectangular cells $\mathcal{C}$ along the vectors $\boldsymbol{e}_1$ and $\boldsymbol{e}_2$ respectively and define $\Delta x := L_{0x}/n_x$, $\Delta y := L_{0y}/n_y$. This means that for each cell $C \in \mathcal{M}$, $m(C) = \Delta x \Delta y$ and for each face $\sigma \in \varepsilon$, $\hat{m}(\sigma) \in \{\Delta x, \Delta y\}$.

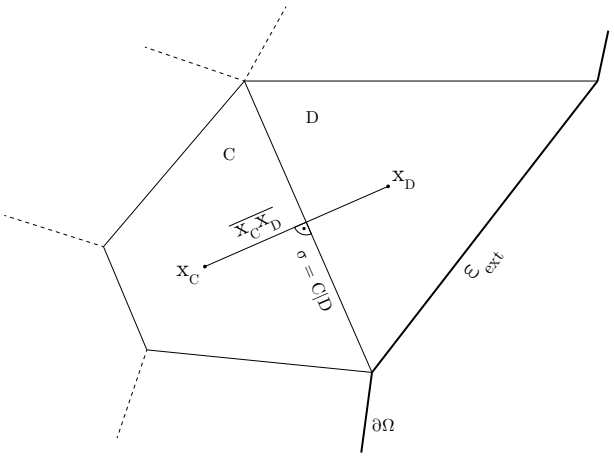


Figure 2.1: Visualisation of the cells C, D in admissible mesh alongside their centroids $\boldsymbol{x}_C, \boldsymbol{x}_D$, their shared boundary $\sigma = C|D$ and their connecting line $\overline{\boldsymbol{x}_C\boldsymbol{x}_D}$. This visualisation was adapted from [SB18].

2.2.1 Finite volume discretization of Laplacian

We derive the approximation of Laplacian using the finite volume formalism. Because of the regular grid, this approximation can be equally obtained using the finite difference method. Let us consider a general function $\Psi : C^2(\Omega) \to \mathbb{R}$ and a cell $C \in \mathcal{C}$. Then

$$\int_C \nabla^2\Psi(\boldsymbol{x})\,\mathrm{d}x = \int_{\partial C} \nabla\Psi(\boldsymbol{x}) \cdot \boldsymbol{n}\,\mathrm{d}S \tag{2.1}$$

$$= \sum_{\sigma\in\varepsilon_C} \int_\sigma \nabla\Psi(\boldsymbol{x}) \cdot \boldsymbol{n}\,\mathrm{d}S \tag{2.2}$$

$$= \sum_{\sigma\in\varepsilon_C, \exists \boldsymbol{x}_\sigma \in \sigma} m(\sigma)\nabla\Psi(\boldsymbol{x}_\sigma) \cdot \boldsymbol{n}_{\boldsymbol{\sigma}} \tag{2.3}$$

where $\boldsymbol{n}_{\boldsymbol{k},\boldsymbol{\sigma}}$ is the normal vector to σ. The divergence theorem was used in (2.1) and the mean value theorem to guarantee existence of such $\boldsymbol{x}_\sigma$ in (2.3).
Next we transition to the mesh function Ψ_d and use

$$\nabla\Psi(\boldsymbol{x}_\sigma) \cdot \boldsymbol{n}_\sigma \approx \frac{\Psi_D - \Psi_C}{\rho(\boldsymbol{x}_C, \boldsymbol{x}_D)} \tag{2.4}$$

for cell D adjacent to C and $\sigma = C|D$. Additionally for quantities $m(\sigma)$ and $\rho(\boldsymbol{x}_C, \boldsymbol{x}_D)$ holds

$$m(\sigma) = \begin{cases} \Delta y \text{ if } \sigma \parallel \boldsymbol{e}_1 \\ \Delta x \text{ if } \sigma \parallel \boldsymbol{e}_2 \end{cases}, \tag{2.5}$$

$$\rho(\boldsymbol{x}_C, \boldsymbol{x}_D) = \begin{cases} \Delta x \text{ if } \sigma \parallel \boldsymbol{e}_1 \\ \Delta y \text{ if } \sigma \parallel \boldsymbol{e}_2 \end{cases}, \tag{2.6}$$

thus expanding (2.3) yields

$$\int_C \nabla^2 \Psi \, \mathrm{d}x \approx \sum_{\sigma \in \varepsilon_C,\ \sigma = C|D} m(\sigma) \frac{\Psi_D - \Psi_C}{\rho(\boldsymbol{x}_C, \boldsymbol{x}_D)} \tag{2.7}$$

$$= \frac{\Delta y}{\Delta x}(\Psi_E - \Psi_C) - \frac{\Delta y}{\Delta x}(\Psi_W - \Psi_C) + \frac{\Delta x}{\Delta y}(\Psi_N - \Psi_C) - \frac{\Delta x}{\Delta y}(\Psi_S - \Psi_C) \tag{2.8}$$

$$= \frac{\Delta y}{\Delta x}(\Psi_E - 2\Psi_C + \Psi_W) + \frac{\Delta x}{\Delta y}(\Psi_N - 2\Psi_C + \Psi_S), \tag{2.9}$$

where $E, W, N, S \in \mathcal{M}$ are cells adjacent to C as shown on Figure 2.2. This expression is only valid for $C \in \mathcal{M}_{\text{int}}$ as it requires all neighbouring cells to exist. We will however use this form on the entire $\mathcal{M}$ by defining additional "ghost cells" so that each cell in $\mathcal{M}$ has all neighbours. We discuss the formulation and values of ghost cells with regards to different boundary conditions in Section 2.2.3.

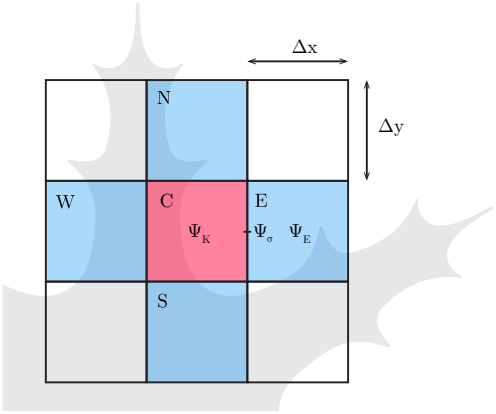


Figure 2.2: Cells around the cell C on regular grid $\mathcal{M}$. The cells are Δx, Δy sized in the $\boldsymbol{e}_1$, $\boldsymbol{e}_2$ directions respectively. The mesh function Ψ is defined at the center of each cell. Ψ_σ is located on the shared boundary σ between cells C and E, written as $\sigma = C|E$. The value of Ψ_σ is obtained by interpolation of the values of Ψ in the adjacent cells Ψ_C and Φ_E. For this simple regular grid linear interpolation is used.

Note that by dividing the equation (2.9) by the volume of the cell C, we obtain

$$\frac{1}{m(C)} \int_C \nabla^2 \Psi \, \mathrm{d}x \approx \frac{\Psi_E - 2\Psi_C + \Psi_W}{\Delta x^2} + \frac{\Psi_N - 2\Psi_C + \Psi_S}{\Delta y^2} =: \nabla_d^2 \Psi_C, \tag{2.10}$$

which is the second order central difference approximation of the Laplacian. We take this approximation as the definition of the discrete Laplacian ∇_d^2 acting on mesh function Ψ_d.

2.2.2 Finite volume discretization of gradient

Repeating a similar procedure as for the approximation of Laplacian, we can write

$$\int_C \nabla\Psi \,\mathrm{d}x = \int_{\partial C} \Psi \cdot \boldsymbol{n} \,\mathrm{d}s = \sum_{\sigma\in\epsilon_C} \int_\sigma \Psi \cdot \boldsymbol{n} \,\mathrm{d}s \tag{2.11}$$

$$= \sum_{\sigma\in\epsilon_C, \exists \boldsymbol{x}_\sigma \in \sigma} m(\sigma)\Psi(\boldsymbol{x}_\sigma)\cdot \boldsymbol{n_\sigma} \tag{2.12}$$

$$\approx \sum_{\sigma\in\epsilon_C} m(\sigma)\Psi_\sigma \cdot \boldsymbol{n_\sigma} \tag{2.13}$$

We employ second order approximation of Ψ_σ for the cell face $\sigma = C|D$

$$\Psi_\sigma = \frac{\rho(\boldsymbol{x}_C,\sigma)\Psi_D + \rho(\boldsymbol{x}_D,\sigma)\Psi_C}{\rho(\boldsymbol{x}_C,\boldsymbol{x}_D)} = \frac{1}{2}(\Psi_D + \Psi_C), \tag{2.14}$$

where the mesh property $\rho(\boldsymbol{x}_C,\sigma) = \frac{1}{2}\rho(\boldsymbol{x}_C,\boldsymbol{x}_D)$ was used.
Expanding (2.13) yields

$$\int_C \nabla\Psi \,\mathrm{d}x \approx \frac{\Delta y}{2}(\Psi_E + \Psi_C)\begin{pmatrix}1\\0\end{pmatrix} + \frac{\Delta y}{2}(\Psi_W + \Psi_C)\begin{pmatrix}-1\\0\end{pmatrix} \tag{2.15}$$

$$+ \frac{\Delta x}{2}(\Psi_N + \Psi_C)\begin{pmatrix}0\\1\end{pmatrix} + \frac{\Delta x}{2}(\Psi_S + \Psi_C)\begin{pmatrix}0\\-1\end{pmatrix} \tag{2.16}$$

$$= \frac{1}{2}\begin{pmatrix}\Delta y(\Psi_E - \Psi_W)\\ \Delta x(\Psi_N - \Psi_S)\end{pmatrix} \tag{2.17}$$

Similar to the discretization of Laplacian, this expression is only valid on $\mathcal{M}_{\text{int}}$ but by utilizing ghost cells we are able to extend it to the entire set of cells $\mathcal{M}$.

Dividing by volume of cell C gives

$$\frac{1}{m(C)}\int_C \nabla\Psi \,\mathrm{d}x \approx \begin{pmatrix}\frac{\Psi_E - \Psi_W}{2\Delta x}\\ \frac{\Psi_N - \Psi_S}{2\Delta y}\end{pmatrix} =: \nabla_d \Psi_C, \tag{2.18}$$

which is the second order central difference approximation of gradient. We take this approximation as the definition of the discrete gradient ∇_d acting on mesh function Ψ_d.

2.2.3 Boundary conditions in the finite volume method

We formulate Dirichlet, Neumann and periodic boundary conditions in a two dimensional finite volume discretization using the so called ghost cells [LeV02]. Set of ghost cells $\mathcal{M}_{\text{ghost}}$ are added along the boundary of the domain Ω such that

$$\forall\sigma \in \varepsilon_{\text{ext}} \exists C \in \mathcal{M}, G \in \mathcal{M}_{\text{ghost}} : \sigma = C|G. \tag{2.19}$$

We will use the same notation as in Section 2.1 for ghost cells $\mathcal{M}_{\text{ghost}}$ as for the cells $\mathcal{M}$. We will extend the domain of all mesh functions Ψ_d to centroids of $\mathcal{M}\cup\mathcal{M}_{\text{ghost}}$.

2.2.3.1 Dirichlet boundary condition

The Dirichlet boundary condition for function $\Psi : \Omega \to \mathbb{R}$ is written as

$$\Psi|_{\partial\Omega} = f, \tag{2.20}$$

where $f : \partial\Omega \to \mathbb{R}$ prescribes values at the boundary.

We wish to express this for mesh function Ψ_d on a single cell face $\sigma \in \varepsilon_{\text{ext}}$. Let $C \in \mathcal{M}, G \in \mathcal{M}_{\text{ghost}}$ such that $\sigma = C|G$. We use the interpolation (2.14) of mesh function Ψ_d for value at the cell face Ψ_σ. The boundary condition becomes

$$\Psi_\sigma = \frac{\rho(\boldsymbol{x}_C, \sigma)\Psi_G + \rho(\boldsymbol{x}_G, \sigma)\Psi_C}{\rho(\boldsymbol{x}_C, \boldsymbol{x}_G)} = f_\sigma, \tag{2.21}$$

where f_σ prescribes values at the cell face. From (2.21) we obtain

$$\Psi_G = \frac{f_\sigma \rho(\boldsymbol{x}_C, \boldsymbol{x}_G) - \rho(\boldsymbol{x}_G, \sigma)\Psi_C}{\rho(\boldsymbol{x}_C, \sigma)}, \tag{2.22}$$

which for the regular grid as defined in Section 2.2 becomes

$$\Psi_G = 2f_\sigma - \Psi_c. \tag{2.23}$$

2.2.3.2 Neumann boundary condition

Reusing the same notation, the Neumann boundary condition states

$$\nabla\Psi \cdot \boldsymbol{n}|_{\partial\Omega} = f, \tag{2.24}$$

where $\boldsymbol{n}$ is the outer normal vector of $\partial\Omega$ in the given point. Using the substitution for gradient operator (2.4) and admissability of mesh $\mathcal{M}$, we can rewrite the boundary condition for mesh function Ψ_d and prescribed value f_σ as

$$\frac{\Psi_G - \Psi_C}{\rho(\boldsymbol{x}_G, \boldsymbol{x}_C)} = f_\sigma, \tag{2.25}$$

thus the value at the ghost cell centroid is

$$\Psi_G = f_\sigma \rho(\boldsymbol{x}_G, \boldsymbol{x}_C) + \Psi_C. \tag{2.26}$$

For regular grid meshes, $\rho(\boldsymbol{x}_G, \boldsymbol{x}_C)$ is equal to Δx resp. Δy, when $\sigma \parallel \boldsymbol{e}_2$ resp. $\sigma \parallel \boldsymbol{e}_1$.

2.2.3.3 Periodic boundary condition

The periodic boundary condition is given only for regions $\Omega = (A_1, B_1) \times (A_2, B_2) \times \ldots \times (A_n, B_n)$, where $n \in \mathbb{N}$ and $\boldsymbol{A}, \boldsymbol{B} \in \mathbb{R}^n$. Let $\boldsymbol{x}, \boldsymbol{y} \in \partial\Omega$ such that $x_i = A_i, y_i = B_i$ for some $i \in \hat{n}$, then the periodic boundary condition states

$$\Psi(\boldsymbol{x}) = \Psi(\boldsymbol{y}). \tag{2.27}$$

That is, the values on the domain boundary $\partial\Omega$ are equal to the values on the opposite boundary.

To formulate this for mesh function Ψ_d, no ghost cells are necessary. Instead, for the regular grid mesh, we define indexing $\forall i \in \hat{n}_y, j \in \hat{n}_x$

$$\Psi_{i,j} := \Psi_d((j-1)\Delta x, (i-1)\Delta y) \tag{2.28}$$

and write the periodic boundary condition as

$$\Psi_{1,j} = \Psi_{n_y,j}, \tag{2.29}$$

$$\Psi_{i,1} = \Psi_{i,n_x}. \tag{2.30}$$

2.2.4 Finite volume discretization of the phase field equations

We use the already derived formulas to discretize the Phase field model. Let $\Phi_d : \mathcal{T} \times \mathcal{P} \to \mathbb{R}$ resp. $T_d : \mathcal{T} \times \mathcal{P} \to \mathbb{R}$ be the mesh function substitution of phase resp. heat field. By integrating both sides of equations (1.19), (1.20) over a cell C, using formulas (2.10), (2.18) and substituting

$$\frac{1}{m(C)} \int_C \frac{\partial \Phi}{\partial t} \mathrm{d}x \approx \frac{\mathrm{d}\Phi_C}{\mathrm{d}t} \tag{2.31}$$

$$\frac{1}{m(C)} \int_C k_1(\nabla \Phi) \mathrm{d}x \approx k_1(\nabla_d \Phi_C) \tag{2.32}$$

$$\frac{1}{m(C)} \int_C k_2(\Phi, \nabla \Phi) \mathrm{d}x \approx k_2(\Phi_C, \nabla_d \Phi_C), \tag{2.33}$$

$$\frac{1}{m(C)} \int_C k_3(\nabla \Phi) \mathrm{d}x \approx k_3(\nabla_d \Phi_C) \tag{2.34}$$

we obtain the semi-discrete scheme of the Phase field model $\forall C \in \mathcal{M}$

$$\frac{\mathrm{d}T_C}{\mathrm{d}t} = \nabla_d^2 T_C + L \frac{\mathrm{d}\Phi_C}{\mathrm{d}t} \qquad \text{in } \mathcal{T}, \tag{2.35}$$

$$\frac{\mathrm{d}\Phi_C}{\mathrm{d}t} = k_1(\nabla_d \Phi_C) \nabla_d^2 \Phi_C + k_0(\Phi_C, \nabla_d \Phi_C) + k_2(\nabla_d \Phi_C)(T^* - T_C) \qquad \text{in } \mathcal{T}, \tag{2.36}$$

$$T_C|_{t=0} = T_{C,\mathrm{ini}}|_{\mathcal{P}}, \tag{2.37}$$

$$\Phi_C|_{t=0} = \Phi_{C,\mathrm{ini}}|_{\mathcal{P}}, \tag{2.38}$$

alongside Dirichlet, Neumann, or periodic boundary conditions for both T_d and Φ_d as specified in Sections 2.2.3.1, 2.2.3.2 and 2.2.3.3.

Chapter 3

Time integration schemes

3.1 Explicit integration schemes

The semi-discrete scheme in Section 2.2.4 forms an initial value problem, for which a solution can be obtained using explicit time integration schemes. We evaluated and compared explicit Euler method, fourth order Runge-Kutta method and fourth order Runge-Kutta-Merson method. We will refer to the resulting schemes utilizing these methods as Euler, RK4 and RKM in order.

In Algorithm 1 we illustrate time integration procedure going from time step n to $n+1$ using the explicit Euler method defined as

$$\frac{\mathrm{d}\Psi^n}{\mathrm{d}t} \approx \frac{\Psi^{n+1} - \Psi^n}{\Delta t}, \tag{3.1}$$

where $\Psi^n := \Psi(n\Delta t)$, $\Psi \in \{\Phi_C, T_C\}$, $C \in \mathcal{M}$ using time step $\Delta t > 0$ and time step index $n \in \mathbb{N}$.

Algorithm 1 Time integration using the explicit Euler method

for each cell $C \in \mathcal{M}$ **do**

$$\nabla_d^2 T_C^n \leftarrow \frac{T_E^n - 2T_C^n + T_W^n}{\Delta x^2} + \frac{T_N^n - 2T_C^n + T_S^n}{\Delta y^2}$$

$$\nabla_d^2 \Phi_C^n \leftarrow \frac{\Phi_E^n - 2\Phi_C^n + \Phi_W^n}{\Delta x^2} + \frac{\Phi_N^n - 2\Phi_C^n + \Phi_S^n}{\Delta y^2}$$

$$\nabla_d \Phi_C^n \leftarrow \begin{pmatrix} \frac{\Phi_E^n - \Phi_W^n}{2\Delta x} \\ \\ \frac{\Phi_N^n - \Phi_S^n}{2\Delta y} \end{pmatrix}$$

$$\frac{\mathrm{d}T_C^n}{\mathrm{d}t} \leftarrow k_1(\nabla_d \Phi_C^n)\nabla_d^2 \Phi_C^n + k_0(\Phi_C^n, \nabla_d \Phi_C^n) + k_2(\nabla_d \Phi_C^n)(T^* - T_C^n)$$

$$\frac{\mathrm{d}T_C^n}{\mathrm{d}t} = \nabla_d^2 T_C^n + L\frac{\mathrm{d}\Phi_C^n}{\mathrm{d}t}$$

$$\Phi_C^{n+1} \leftarrow \Phi_C^n + \Delta t \frac{\mathrm{d}T_C^n}{\mathrm{d}t}$$

$$T_C^{n+1} \leftarrow T_C^n + \Delta t \frac{\mathrm{d}T_C^n}{\mathrm{d}t}$$

end for

3.1.1 Runge-Kutta-Merson method

For completeness we include algorithm of the fourth order Runge-Kutta-Merson method [Chr70] and [Str12]. This method is used to solve system of differential equations in the form

$$\frac{\partial \boldsymbol{x}}{\partial t} = \boldsymbol{f}(t, \boldsymbol{x}), \tag{3.2}$$

where $\boldsymbol{x} \in \mathbb{R}^N$ is the unknown vector function and $\boldsymbol{f} : \mathcal{T} \times \mathbb{R}^N \to \mathbb{R}^N$ is the function of right hand side with $N \in \mathbb{N}$. The Runge-Kutta-Merson algorithm is shown in Algorithm 2 below.

Algorithm 2 Runge-Kutta-Merson method

Let $\boldsymbol{f} : \mathbb{R}^{1+N} \to \mathbb{R}^N$ be the numerically integrated function with $N \in \mathbb{N}$.
Let $\Delta t_{ini} \in \mathbb{R}$ resp. $\boldsymbol{x}_{ini} \in \mathbb{R}^N$ be the initial timestep resp. initial condition.
Let $t \in \mathbb{R}$ be the current time level, $T \in \mathbb{R}$ final time and $\varepsilon \in \mathbb{R}$ target integration error.
$\Delta t = \Delta t_{ini}$
$\boldsymbol{x} = \boldsymbol{x}_{ini}$
loop
  if $T - t < \Delta t$ **then**
    $\Delta t = T - t$
  end if
  $\boldsymbol{K}_1 = \boldsymbol{f}(t, \boldsymbol{x})$
  $\boldsymbol{K}_2 = \boldsymbol{f}(t + \frac{\Delta t}{3}, \boldsymbol{x} + \frac{\Delta t}{3}\boldsymbol{K}_1)$
  $\boldsymbol{K}_3 = \boldsymbol{f}(t + \frac{\Delta t}{3}, \boldsymbol{x} + \frac{\Delta t}{6}(\boldsymbol{K}_1 + \boldsymbol{K}_2))$
  $\boldsymbol{K}_4 = \boldsymbol{f}(t + \frac{\Delta t}{2}, \boldsymbol{x} + \frac{\Delta t}{8}(\boldsymbol{K}_1 + 3\boldsymbol{K}_2))$
  $\boldsymbol{K}_5 = \boldsymbol{f}(t + \Delta t, \boldsymbol{x} + \frac{\Delta t}{2}(\boldsymbol{K}_1 - 3\boldsymbol{K}_3 + 4\boldsymbol{K}_4))$
  $\delta = \frac{\Delta t}{3}\|0.2\boldsymbol{K}_1 - 0.9\boldsymbol{K}_3 + 0.8\boldsymbol{K}_4 - 0.1\boldsymbol{K}_5\|_\infty$
  if $\delta < \varepsilon$ **then**
    $\boldsymbol{x} \leftarrow \boldsymbol{x} + \frac{\Delta t}{6}(\boldsymbol{K}_1 + 4\boldsymbol{K}_4 + \boldsymbol{K}_5)$
    $t \leftarrow t + \Delta t$
    if $t \geq T$ **then**
      break
    end if
  end if
  if $\delta > 0$ **then**
    $\Delta t \leftarrow \left(\frac{\varepsilon}{\delta}\right)^{0.2} \cdot \Delta t$
  end if
end loop

3.2 Semi-implicit integration scheme

We derive semi-implicit time integration scheme of the semi-discrete scheme (S-I) given in Section 2.2.4 using the Crank-Nicolson method [CN47]. One advatnage of the Crank-Nicolson method over the previously described explicit methods is its unconditional stability [Tho98].

3.2.1 Coupled integration scheme

For simplicity of notation, let us denote $\Phi := \Phi_C$, $\nabla\Phi := \nabla_d\Phi_C$, $\nabla^2\Phi := \nabla_d^2\Phi_C$ for some cell C. Similarly $T := T_C$, $\nabla^2 T := \nabla_d^2 T_C$.

First we time discretize the equations (1.19), (1.20) by applying more general form of the Crank-Nicolson time discretization given by

$$\frac{\Psi^{n+1} - \Psi^n}{\Delta t} \approx \gamma \frac{\partial \Psi^{n+1}}{\partial t} + (1-\gamma)\frac{\partial \Psi^n}{\partial t} \tag{3.3}$$

for $\gamma \in [0,1]$. By setting $\gamma = 0$ we obtain explicit Euler method, by setting $\gamma = 1/2$ we obtain the Crank-Nicolson method [CN47] and by setting $\gamma = 1$ the implicit Euler method. Applying this discretization to the semi-discrete scheme in Section 2.2.4 yields

$$\frac{T^{n+1} - T^n}{\Delta t} \approx \gamma \left[\nabla^2 T^{n+1} + L\frac{\partial \Phi^{n+1}}{\partial t}\right] + (1-\gamma)\left[\nabla^2 T^n + L\frac{\partial \Phi^n}{\partial t}\right], \tag{3.4}$$

$$\begin{aligned}\frac{\Phi^{n+1} - \Phi^n}{\Delta t} \approx{}& \gamma\left[k_1(\nabla\Phi^{n+1})\nabla^2\Phi^{n+1} + k_0(\Phi^{n+1}, \nabla\Phi^{n+1}) + k_2(\nabla\Phi^{n+1})(T^* - T^{n+1})\right] \\ &+ (1-\gamma)\left[k_1(\nabla\Phi^n)\nabla^2\Phi^n + k_0(\Phi^n, \nabla\Phi^n) + k_2(\nabla\Phi^n)(T^* - T^n)\right].\end{aligned} \tag{3.5}$$

Since k_0, k_1, k_2 are nonlinear, we cannot evaluate them at point Φ^{n+1}, thus an approximation $k_i(\Phi^{n+1}) \approx k_i(\Phi^n)$ is used for $i \in \{1,2,3\}$. Because now all k_i functions are evaluated in the same timestep, we drop the argument notation and denote $k_0 := k_0(\Phi^n, \nabla\Phi^n)$, $k_1 := k_1(\nabla\Phi^n)$, $k_2 := k_2(\nabla\Phi^n)$. Thus (3.5) becomes

$$\begin{aligned}\frac{\Phi^{n+1} - \Phi^n}{\Delta t} ={}& \gamma\left[k_1\nabla^2\Phi^{n+1} + k_0 + k_2(T^* - T^{n+1})\right] \\ &+ (1-\gamma)\left[k_1\nabla^2\Phi^n + k_0 + k_2(T^* - T^n)\right].\end{aligned} \tag{3.6}$$

By multiplying (3.6) by Δx and isolating all terms containing T^{n+1}, Φ^{n+1} in (3.6), (3.4) we obtain

$$T^{n+1} - \Delta t\gamma(\nabla^2 T^{n+1} + L\frac{\partial \Phi^{n+1}}{\partial t}) = T^n + \Delta t(1-\gamma)(\nabla^2 T^n + L\frac{\partial \Phi^n}{\partial t}), \tag{3.7}$$

$$\Phi^{n+1} - \Delta t\gamma(k_1\nabla^2\Phi^{n+1} - k_2 T^{n+1}) = \Phi^n + \Delta t(1-\gamma)(k_1\nabla^2\Phi^n - k_2 T^n) + \Delta t(k_0 + k_2 T^*). \tag{3.8}$$

Substituting $\frac{\partial\Phi^{n+1}}{\partial t} \approx \frac{\Phi^{n+1}-\Phi^n}{\gamma\Delta t} - \frac{1-\gamma}{\gamma}\frac{\partial\Phi^n}{\partial t}$ in equation (3.7) derived from (3.3), and shuffling terms yields

$$T^{n+1} - \Delta t\gamma\nabla^2 T^{n+1} - L\Phi^{n+1} = T^n - L\Phi^n + \Delta t(1-\gamma)\nabla^2 T^n. \tag{3.9}$$

Now we transition from a notation in terms of general cell C to using its indices $(i,j) \in \hat{n}_y \times \hat{n}_x$. Let $\boldsymbol{P} \in \mathbb{R}^N$ with $N := n_x n_y$ be vector representation of the mesh function $\Psi : \mathcal{M} \to \mathbb{R}$ such that $\forall j \in \hat{n}_x, i \in \hat{n}_y$: $\boldsymbol{P}_{j+n_x(i-1)} = \Psi((j-1)\Delta x, (i-1)\Delta x) = \Psi_{i,j} = \Psi_C$. Using the discrete

Laplacian $\nabla_d^2\Psi$ (2.10), discrete gradient $\nabla_d\Psi$ (2.18) and describing cells E, W, N, S using indices with respect to cell at $(i,j) \in \hat{n}_y \times \hat{n}_x$ we define its vector equivalent

$$\begin{aligned}\nabla_d^2 \boldsymbol{P}_{j+n_x(i-1)} := &\frac{\boldsymbol{P}_{j+1+n_x(i-1)} - 2\boldsymbol{P}_{j+n_x(i-1)} + \boldsymbol{P}_{j-1+n_x(i-1)}}{\Delta x^2} \\ &+ \frac{\boldsymbol{P}_{j+n_x i} - 2\boldsymbol{P}_{j+n_x(i-1)} + \boldsymbol{P}_{j+n_x(i-2)}}{\Delta y^2},\end{aligned} \tag{3.10}$$

$$\nabla_d \boldsymbol{P}_{j+n_x(i-1)} := \begin{pmatrix} \frac{P_{j+1+n_x(i-1)} - P_{j-1+n_x(i-1)}}{2\Delta x} \\ \frac{P_{j+n_x i} - P_{j+n_x(i-2)}}{2\Delta y} \end{pmatrix}. \tag{3.11}$$

We define $\boldsymbol{F}, \boldsymbol{U} \in \mathbb{R}^N$ such that $\forall j \in \hat{n}_x, i \in \hat{n}_y$

$$\boldsymbol{F}_{j+n_x(i-1)} = \Phi_{i,j}, \tag{3.12}$$

$$\boldsymbol{U}_{j+n_x(i-1)} = T_{i,j}, \tag{3.13}$$

where $\Phi_{i,j}, T_{i,j}$ are given by (2.28).

Rewriting equations (3.14) and (3.8) in terms of $\boldsymbol{U}$ and $\boldsymbol{F}$ using the equations (3.10), for ∇_d^2 and (3.11) for ∇_d yields

$$\boldsymbol{U}^{n+1} - \Delta t\gamma\nabla_d^2\boldsymbol{U}^{n+1} - L\boldsymbol{F}^{n+1} = \boldsymbol{U}^n - L\boldsymbol{F}^n + \Delta t(1-\gamma)\nabla_d^2\boldsymbol{U}^n, \tag{3.14}$$

$$\boldsymbol{F}^{n+1} - \Delta t\gamma(k_1\nabla_d^2\boldsymbol{F}^{n+1} - k_2T^{n+1}) = \boldsymbol{F}^n + \Delta t(1-\gamma)(k_1\nabla_d^2\boldsymbol{F}^n - k_2T^n) + \Delta t(k_0 + k_2T^*), \tag{3.15}$$

which can be expressed as a system of linear equations in the form

$$\begin{pmatrix} A_F & B_F \\ B_U & A_U \end{pmatrix}\begin{pmatrix} \boldsymbol{F} \\ \boldsymbol{U} \end{pmatrix} = \begin{pmatrix} \boldsymbol{b}_F \\ \boldsymbol{b}_U \end{pmatrix} \tag{3.16}$$

with the values of A_F, B_F, B_U, $A_U \in \mathbb{R}^{N\times N}$ and vectors of right hand side $\boldsymbol{b}_F$, $\boldsymbol{b}_U \in \mathbb{R}^N$ given

by $\forall(k,l) \in \hat{N} \times \hat{N}$ by

$$(A_F)_{k,l} = \begin{cases} 1 + k_1(\frac{2\gamma\Delta t}{\Delta x^2} + \frac{2\gamma\Delta t}{\Delta y^2}) & \text{when } l = k, \\ -k_1\frac{\gamma\Delta t}{\Delta x^2} & \text{when } l+1 = k, \\ -k_1\frac{\gamma\Delta t}{\Delta x^2} & \text{when } l-1 = k, \\ -k_1\frac{\gamma\Delta t}{\Delta y^2} & \text{when } l+n_x = k, \\ -k_1\frac{\gamma\Delta t}{\Delta y^2} & \text{when } l-n_x = k, \\ 0 & \text{otherwise} \end{cases} \tag{3.17}$$

$$(B_F)_{k,l} = \begin{cases} -\gamma\Delta t k_2 & \text{when } l = k, \\ 0 & \text{otherwise} \end{cases} \tag{3.18}$$

$$(A_U)_{k,l} = \begin{cases} 1 + \frac{2\gamma\Delta t}{\Delta x^2} + \frac{2\gamma\Delta t}{\Delta y^2} & \text{when } l = k, \\ -\frac{\gamma\Delta t}{\Delta x^2} & \text{when } l+1 = k, \\ -\frac{\gamma\Delta t}{\Delta x^2} & \text{when } l-1 = k, \\ -\frac{\gamma\Delta t}{\Delta y^2} & \text{when } l+n_x = k, \\ -\frac{\gamma\Delta t}{\Delta y^2} & \text{when } l-n_x = k, \\ 0 & \text{otherwise} \end{cases} \tag{3.19}$$

$$(B_U)_{k,l} = \begin{cases} -L & \text{when } l = k, \\ 0 & \text{otherwise} \end{cases} \tag{3.20}$$

$$(\boldsymbol{b}_F)_k = \boldsymbol{F}_k^n + \Delta t(1-\gamma)(k_1\nabla_d^2\boldsymbol{F}_k^n - k_2\boldsymbol{U}_k^n) + \Delta t(k_0 + k_2T^*), \tag{3.21}$$

$$(\boldsymbol{b}_U)_k = \boldsymbol{U}_k^n - L\boldsymbol{F}_k^n + \Delta t(1-\gamma)\nabla_d^2\boldsymbol{U}_k^n. \tag{3.22}$$

In (3.17), the argument $\nabla_d^2\boldsymbol{F}_k^n$ for the k_1, k_2, k_3 functions was for readability of the expressions elided.

Such system of linear equations can be solved by an appropriate matrix solver such as the Successive over-relaxation method (SOR) [You50]. However, in practice, with physically accurate choice of simulation parameters, this system is ill fit for numerical solvers. Observe that in the off diagonal matrix B_U the only non-zero values are $-L$ without dependence on the mesh resolution Δx or time step Δt. In our experiments we choose $L = 2$ making the matrix not diagonally dominant and causing the employed solver to diverge, regardless of choice for other model parameters.

3.2.2 Split integration scheme

The poor convergence of the coupled matrix scheme motivates us to attempt at least an approximate solution. One approach is to use the operator splitting method [MS16]. The main idea is to split the complex problem into simpler sub-problems, solve each individually, then combine their solutions to obtain the final solution. In our case by solving the system of equations for the phase field and temperature field separately.

We begin with the equations (3.14) and (3.15) and approximate the $\boldsymbol{U}^{n+1}$ term with $\boldsymbol{U}^n$ in the $\boldsymbol{F}$ equation then move it to the right side. Thus a new equation is obtained for the right hand side

$$(\boldsymbol{b}_F)_k = \boldsymbol{F}_k^n + \Delta t(1-\gamma)k_1\nabla_d^2\boldsymbol{F}_k^n + \Delta t(k_0 + k_2(T^* - \boldsymbol{U}_k^n)) \qquad \forall k \in \hat{N}. \tag{3.23}$$

This modification enables

$$A_F^n \boldsymbol{F}^{n+1} = \boldsymbol{b}_F^n \tag{3.24}$$

to be solved independently using only the current state $\boldsymbol{F}^n$ and yielding the new state $\boldsymbol{F}^{n+1}$. Since $\boldsymbol{F}^{n+1}$ is now known, we can move it to the right hand side of the equation (3.14) giving us modified equation for its right hand side

$$(b_U)_k^{n,n+1} = \boldsymbol{U}_k^n - L(\boldsymbol{F}_k^{n+1} - \boldsymbol{F}_k^n) + \Delta t(1-\gamma)\nabla_d^2 \boldsymbol{U}_k^n \qquad \forall k \in \hat{N}. \tag{3.25}$$

which allows the new temperature field $\boldsymbol{U}^{n+1}$ to be solved for in the matrix equation

$$A_U^n \boldsymbol{U}^{n+1} = \boldsymbol{U_F}^{n,n+1}. \tag{3.26}$$

Notice that both A_F^n, A_U^n are positively definite, symmetric matrices and with an appropriate choice of $\Delta x, \Delta y, \Delta t$ also diagonally dominant. This guarantees convergence for many matrix solvers and specifically allows the Conjugate gradient method to be used, which is given in Algorithm 3.

3.2.2.1 Conjugate gradient method

For completeness, we include the Conjugate gradient method algorithm [She94]. Conjugate gradient method is used to solve matrix equations in the form

$$A\boldsymbol{x} = \boldsymbol{b}, \tag{3.27}$$

where $\boldsymbol{x} \in \mathbb{R}^N$ is the vector of unknowns, $A \in \mathbb{R}^{N\times N}$ is symmetric, positive-semidefinite and $\boldsymbol{b} \in \mathbb{R}^N$. It is well suited for large sparse matrices such as those arising from the finite difference and finite element methods [She94]. Since all operations are performed using vectors, it is amenable to parallel implementation.

Algorithm 3 Conjugate Gradient method

Let $A \in \mathbb{R}^{N\times N}$, $\boldsymbol{b} \in \mathbb{R}^N$ represent a system of linear equations with $N \in \mathbb{N}$.
Let $\boldsymbol{x}^0$ be initial guess at the solution and $\varepsilon \in \mathbb{R}$ tolerence for the residual
Let $M \in \mathbb{N}$ be the maximum number of iterations.
The output is $\boldsymbol{x} \in \mathbb{R}^N$ such that $\|A\boldsymbol{x} - \boldsymbol{b}\|_\infty < \varepsilon$.
$\boldsymbol{d}^0 = \boldsymbol{r}^0 = \boldsymbol{b} - A\boldsymbol{x}^0$
for $k = 0, \ldots, M$ **do**
  if $\left\|\boldsymbol{r}^k\right\|_\infty < \varepsilon$ or $k > M$ **then**
    $\boldsymbol{x} = \boldsymbol{x}^k$
    break
  end if
  $\alpha^k = \dfrac{\boldsymbol{r}^k \cdot \boldsymbol{r}^k}{(\boldsymbol{d}^k)^T A\boldsymbol{d}^k}$
  $\boldsymbol{x}^{k+1} = \boldsymbol{x}^k + \alpha^k \boldsymbol{d}^k$
  $\boldsymbol{r}^{k+1} = \boldsymbol{r}^k + \alpha^k A\boldsymbol{d}^k$
  $\beta^k = \dfrac{\boldsymbol{r}^{k+1} \cdot \boldsymbol{r}^{k+1}}{\boldsymbol{r}^k \cdot \boldsymbol{r}^k}$
  $\boldsymbol{d}^{k+1} = \boldsymbol{r}^{k+1} + \beta^{k+1} A\boldsymbol{d}^k$
end for

3.3 Operator splitting corrections

We can rewrite the explicit and semi-implicit schemes in the form

$$\boldsymbol{F}^{n+1} = S_F(\boldsymbol{F}^n, \boldsymbol{U}^n), \tag{3.28}$$
$$\boldsymbol{U}^{n+1} = S_U(\boldsymbol{F}^{n+1}, \boldsymbol{U}^n), \tag{3.29}$$

where $\boldsymbol{F}^n, \boldsymbol{U}^n \in \mathbb{R}^N$ are vectors of the phase resp. temperature field mesh functions Φ, T, given by (3.12) and (3.13) at timestep $n \in \mathbb{N}$. $S_F, S_U : \mathbb{R}^N \times \mathbb{R}^N \to \mathbb{R}^N$ are functions encompassing the explicit and semi-implicit time integration schemes (see Sections 3.2 and 3.1) used to calculate the next iteration of $\boldsymbol{F}^n, \boldsymbol{U}^n$ respectively.

For the explicit schemes the definition of S_F, S_U can be given explicitly using the vectors $\boldsymbol{F}^n, \boldsymbol{U}^n$. For the semi-implicit scheme, S_F, S_U are defined in terms of solution to the matrix equations (3.24), (3.26).

Notice that the calculation of $\boldsymbol{F}^{n+1}$ uses $\boldsymbol{U}^n$, while the calculation $\boldsymbol{U}^{n+1}$ uses $\boldsymbol{F}^{n+1}$. Compared to equations in the semi-discrete scheme given in Section 2.2.4

$$\frac{\mathrm{d}T_C}{\mathrm{d}t} = \nabla_d^2 T_C + L\frac{\mathrm{d}\Phi_C}{\mathrm{d}t},$$
$$\frac{\mathrm{d}\Phi_C}{\mathrm{d}t} = k_1(\nabla_d \Phi_C)\nabla_d^2 \Phi_C + k_0(\Phi_C, \nabla_d \Phi_C) + k_2(\nabla_d \Phi_C)(T^* - T_C),$$

where Φ depends on T and vice versa at any point in time $t \in \mathcal{T}$. In other words, there is bi-directional dependency between the two equations, but the current time integration schemes only models one direction.

To quantify the error commited by the operator splitting techniques we define n-th *step residual* $r_\Phi^n \in \mathbb{R}^N$ for both categories of time integration schemes as

$$r_\Phi^n := S_F(\boldsymbol{F}^n, \boldsymbol{U}^n) - S_F(\boldsymbol{F}^n, \boldsymbol{U}^{n+1}), \tag{3.30}$$

which can be read as "the difference between vector $\boldsymbol{F}^n$ obtained using the information from the current time step and the vector$\boldsymbol{\mathcal{F}}$ we would have obtained the temperature field vector $\boldsymbol{U}^{n+1}$ from the next time step was used". Note that r_Φ^n quantifies only the internal consistency of the approximations and makes no statement about the quality of the chosen approximations in relation to the exact solution of the original equations.
Using the scheme functions (3.28) we can rewrite (3.30) as

$$\begin{aligned} r_\Phi^n &= \boldsymbol{F}^{n+1} - S_F(\boldsymbol{F}^n, \boldsymbol{U}^{n+1}) \\ &= \boldsymbol{F}^{n+1} - S_F(\boldsymbol{F}^n, S_U(\boldsymbol{F}^{n+1}, \boldsymbol{U}^n)) \end{aligned}$$

and generalize its definition for a candidate vector $\boldsymbol{\mathcal{F}} \in \mathbb{R}^N$ for the next step $\boldsymbol{F}^{n+1}$ as

$$r_\Phi = \boldsymbol{\mathcal{F}} - S_F(\boldsymbol{F}^n, S_U(\boldsymbol{\mathcal{F}}, \boldsymbol{U}^n)). \tag{3.31}$$

3.3.1 Repeated time stepping

We can lower the magnitude of the step residual r_Φ, thus potentially increasing the accuracy of our model, by iterating the candidate $\boldsymbol{\mathcal{F}}$ for the next step $\boldsymbol{F}^{n+1}$ repeatedly until the norm of step residual r_Φ^n falls below desired accuracy. The Algorithm 4 shows the procedure going from time step n to $n+1$, while keeping $\|r_\Phi^n\|$ small.

Algorithm 4 Repeated time stepping correction

$\boldsymbol{\mathcal{F}}^0 \leftarrow \boldsymbol{F}^n$
$\boldsymbol{\mathcal{U}}^0 \leftarrow \boldsymbol{U}^n$
for $k = 1, \ldots, M$ **do**
  $\boldsymbol{\mathcal{F}} \leftarrow S_F(\boldsymbol{F}^n, \boldsymbol{\mathcal{U}}^{k-1})$
  $\boldsymbol{\mathcal{U}} \leftarrow S_U(\boldsymbol{\mathcal{F}}, \boldsymbol{U}^n)$
  $r_\Phi \leftarrow \boldsymbol{\mathcal{F}}^{k-1} - \boldsymbol{\mathcal{F}}$
  if $\|r_\Phi\| < \varepsilon$ **then**
    $\boldsymbol{\mathcal{F}}^k \leftarrow \boldsymbol{\mathcal{F}}$
    $\boldsymbol{\mathcal{U}}^k \leftarrow \boldsymbol{\mathcal{U}}$
    break
  end if
  $\boldsymbol{\mathcal{F}}^k \leftarrow (1-\alpha)\boldsymbol{\mathcal{F}}^{k-1} + \alpha\boldsymbol{\mathcal{F}}$
  $\boldsymbol{\mathcal{U}}^k \leftarrow (1-\alpha)\boldsymbol{\mathcal{U}}^{k-1} + \alpha\boldsymbol{\mathcal{U}}$
end for
$\boldsymbol{F}^{n+1} \leftarrow \boldsymbol{\mathcal{F}}^k$
$\boldsymbol{U}^{n+1} \leftarrow \boldsymbol{\mathcal{U}}^k$

In Algorithm 4, $\varepsilon \in \mathbb{R}$ is the target step residual norm, $M \in \mathbb{N}$ is maximum number of iterations and $\alpha \in (0, 1]$ is a relaxation parameter to ensure convergence. Notice that for $\alpha = 1$ the formula $r_\Phi = \boldsymbol{\mathcal{F}}^{k-1} - \boldsymbol{\mathcal{F}}$ in the algorithm, matches the definition (3.31) with $\boldsymbol{\mathcal{F}}^{k-1}$ as the current candidate. This is because $\boldsymbol{\mathcal{F}} = S_F(\boldsymbol{F}^n, S_U(\boldsymbol{\mathcal{F}}^{k-1}, \boldsymbol{U}^n))$.

From our testing with $\alpha = 1$ the Algorithm 4 drives down the step residual arbitrarily r_Φ^n low. However, for each iteration we have to perform the entire solution procedure anew. This means that when using the Algorithm 4 and performing M iterations, the total runtime will be M times longer then if the algorithm was not used.

3.3.2 Correction term

We can achieve similar results to r_Φ for almost no additional cost by making the the Φ equation "more implicit". We start by considering the explicit Euler time discretization of the heat equation (1.19)

$$\frac{T^{n+1} - T^n}{\Delta t} \approx \nabla^2 T^n + L\frac{\partial \Phi^n}{\partial t} \tag{3.32}$$

$$\tag{3.33}$$

and factor out T^{n+1}

$$T^{n+1} = T^n + \Delta t \nabla^2 T^n + \Delta t L \frac{\partial \Phi^n}{\partial t}. \tag{3.34}$$

$$\tag{3.35}$$

Substituting T^{n+1} for T in the phase field equation (1.20) we obtain

$$\frac{\partial \Phi}{\partial t} = k_1(\nabla\Phi)\nabla^2\Phi + k_0(\Phi, \nabla\Phi) + k_2(\nabla\Phi)(T^* - T - \Delta t\nabla^2 T - \Delta t L\frac{\partial \Phi}{\partial t}) \tag{3.36}$$

We dropped the T^n notation in favor of T since we consider the resulting equation a brand new object instead of time discretized version of the equation (1.20).
Rearranging so that we isolate $\frac{\partial \Phi}{\partial t}$ yields

$$\frac{\partial \Phi}{\partial t} = \frac{k_1(\nabla\Phi)\nabla^2\Phi + k_0(\Phi, \nabla\Phi) + k_2(\nabla\Phi)(T^* - T - \Delta t\nabla^2 T)}{1 + \Delta t L k_2(\nabla\Phi)} \tag{3.37}$$

By defining rescaled nonlinear functions

$$\tilde{k}_0(\Phi, \nabla\Phi) := \frac{k_0(\Phi, \nabla\Phi)}{1 + \Delta t L k_2(\nabla\Phi)}, \quad \tilde{k}_1(\nabla\Phi) := \frac{k_1(\nabla\Phi)}{1 + \Delta t L k_2(\nabla\Phi)}, \quad \tilde{k}_2(\nabla\Phi) := \frac{k_2(\nabla\Phi)}{1 + \Delta t L k_2(\nabla\Phi)}, \tag{3.38}$$

we can rewrite the modified equation into a form very closely matching the original (1.20)

$$\frac{\partial \Phi}{\partial t} = \tilde{k}_1(\nabla\Phi)\nabla^2\Phi + \tilde{k}_0(\Phi, \nabla\Phi) + \tilde{k}_2(\nabla\Phi)(T^* - T - \Delta t\nabla^2 T) \tag{3.39}$$

Upon transition to mesh functions and discrete operators this combined with the temperature filed equation (2.36) forms a different "corrected" semi-discrete scheme. Note that the constant Δt in this semi-discrete scheme only makes sense in context of time integration schemes that are performed without intermediate time steps, that is the explicit Euler scheme and semi-implicit scheme. Still, all previously discussed time integration schemes can be rederived for this corrected scheme and behave consistently.

For the explicit schemes the change can be derived trivially by. Its worth noting that all of the additional terms in (3.37) were already being calculated in each time step, thus this "corrected" version has negligible added simulation runtime cost.

To derive the "corrected" semi-implicit time integration scheme we perform the same steps as when deriving the semi-discrete scheme in Section 3.2. The resulting matrices A_F^n, A_U^n are the same as (3.17) but k_1, k_2 and k_3 are replaced with their rescaled counterparts $\tilde{k}_1$, $\tilde{k}_2$ and $\tilde{k}_3$.

Right hand side vector $\boldsymbol{b}_F$ is also rescaled compared to (3.23) and contains an additional term from the added $-\tilde{k}_2 \Delta t \nabla^2 T$ yielding $\forall k \in \hat{N}$

$$(\boldsymbol{b}_F)_k = \boldsymbol{F}_k^n + \Delta t(1-\gamma) k_1 \nabla_d^2 \boldsymbol{F}_k^n + \Delta t(k_0 + k_2(T^* - \boldsymbol{U}_k^n - \Delta t \nabla_d^2 \boldsymbol{U}_k^n)) \tag{3.40}$$

where we dropped the argument notation for the $\tilde{k}_1$, $\tilde{k}_2$, $\tilde{k}_3$ functions. This additional term has to be calculated adding non insignificant simulation runtime cost. However in practice its addition makes the employed matrix solver converge in fewer iterations, causing a reduction in total runtime.

Chapter 4

Implementation

In this chapter we describe algorithms used for the simulation implementation using CUDA. First we introduce the CUDA programming model and its relation to CUDA hardware. Then we detail efficient implementation of few significant GPU algorithms used, in particular parallel for, parallel tiled for and parallel reduction. Finally we perform a benchmark showcasing the efficiency of one of the proposed algorithms. The simulation implementation, examples in this text as well as their tests are publicly available in the project repository.

4.1 CUDA programming model

GPU hardware can provide vastly superior instruction throughput and memory bandwidth compared to CPUs for highly parallel workloads. GPUs achieve this by prioritizing parallel execution of code with small amount of control flow and amortizing latency with the number of items processed. This is to be compared with CPUs which specialize in fast serial, low-latency execution of complex control flow logic [Cor24]. These design trade-offs pose unique programming challenges related to partitioning work amenably to highly parallel nature of the GPU and ensuring such partitioning remains efficient independent on the specific GPU hardware. CUDA programming model aims to solve these challenges while providing fine-grained control over the target hardware and remaining easy to integrate into high-level programming languages such as C++, FORTRAN or Python.

CUDA integrates into the C/C++ ecosystem by providing small set of language extensions which enable GPU programs to be intermixed with code executed on the CPU. These small GPU programs are called *kernels*. Upon launching kernels are scheduled on the GPU in a hierarchy of granularity, each level enabling different level of control. The smallest level of granularity is a thread. The programmer writes code in the Single Instruction Multiple Thread model (SIMT), that is as if for just one thread, while the driver efficiently applies this code to all threads participating in the kernel launch. The next level of granularity is *thread block*, which is a collection of executing threads. Threads can communicate within a single block using *shared memory* and synchronize using barriers. All threads in a single block execute on a single *Streaming Multiprocessor* (SM) but multiple blocks are scheduled and executed independently by the CUDA runtime (unless grouped in block clusters). All blocks corresponding to one kernel are grouped into conceptual *grid* [Cor24].

CUDA source code is complied using the NVCC (Nvidia CUDA C) compiler, which divides the code into kernel and non-kernel functions. Then NVCC compiles kernel code into `PTX` byte code, understandable by CUDA GPU drivers. Non-kernel functions are compiled via user chosen

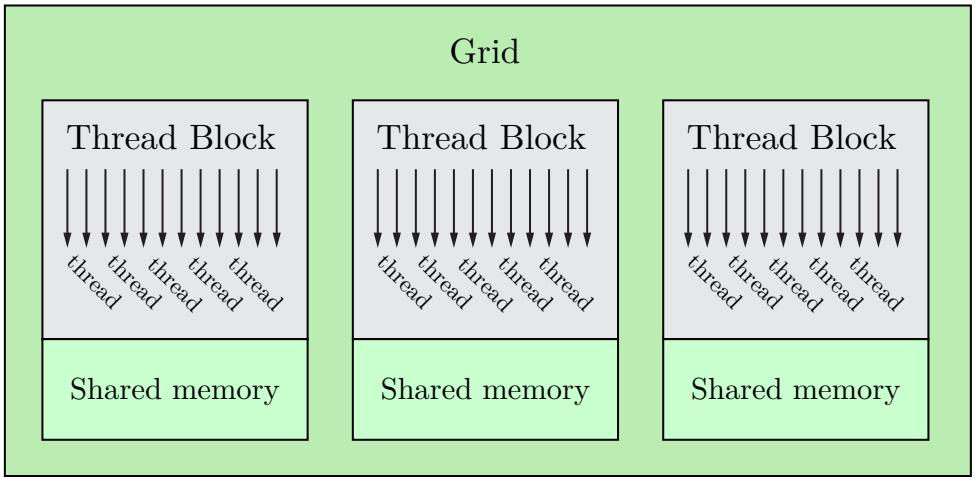


Figure 4.1: Levels of parallelism granularity. Threads are grouped into thread blocks, which are grouped into conceptual grid. All threads of a thread block have access to its shared memory.

host compiler, for example `g++`, translating the code into an object file compatible with the C ecosystem. Kernel launches and other CUDA specific functions are translated into CUDA runtime function calls. When the resulting program is run, kernel PTX byte code is just-in-time translated into assembly code specific to the used GPU. This mechanism provides additional hardware independence and enables post-launch performance improvements via software updates [Cor24].

In Listing 1 is demonstrated a simple "saxpy" cuda kernel performing the vector operation $a\boldsymbol{x} + \boldsymbol{y}$.

```
__global__ void saxpy(float* result, float a, const float* x, const float* y)
{
    int i = threadIdx.x;
    result[i] = a*x[i] + y[i];
}

int main()
{
    float* result = ...
    float* x = ...
    float* y = ...
    float a = ...;
    int N = 1000;

    saxpy<<<1, N>>>(result, a, x, y);
    assert(cudaGetLastError() == 0);
}
```

Listing 1: Naive saxpy kernel

On line 1 of Listing 1, a kernel is declared using the `__global__` keyword. Inside of a kernel we have access to built-in variables provided by the execution engine. One of them is `threadIdx` which denotes the global index of the currently executing thread. For convenient mapping to two or three dimensional problems this index is given as three dimensional vector of type `dim3`. Because our program is one dimensional, on line 3, we access only its `x` component to get the index on which to perform computation. On line 4, we perform the parallel computation in the SIMT model. On line 15, we launch the kernel with CUDA launch syntax `<<<1, N>>>`. The first *launch parameter* (`1`) is the *grid size*, the number of blocks to launch, the second (`N`) is *block size*, the number of threads to use for all blocks. These can alternatively be given as type `dim3` to specify the size in three dimensions. On line 16, we assert that no error occurred using the `cudaGetLastError()` function. This function returns a standardized error code type `cudaError_t` whose value of 0 corresponds to no error [Cor24]. We will discuss the variable initialization on lines 9 – 12, in the subsection 4.1.1.

4.1.1 CUDA memory

CUDA divides the memory space available to the program into a few specific types. We will only discuss the ones needed by our implementation. Broadly, memory is split into *host memory* and *device memory*. Here *host* means the CPU issuing kernel launches on one or more CUDA enabled GPU *devices*. As the name implies, *host memory* resides in the hosts RAM while *device memory* resides in device's VRAM (assuming mapped). Data needs to be explicitly allocated and transferred between host and device using specific functions provided by the CUDA runtime. Additionally, *unified memory* can be used, in which case CUDA will transfer data automatically. While unified memory can provide certain advantages related to performance and developer productivity, it is unsuitable for beginner friendly introduction, and thus it will not be discuss further [Cor24].

In Listing 2, there is an example of device memory vector initialization to sequence of increasing numbers and subsequent cleanup. Such initialization can be used for all vectors in the "saxpy" Listing 1.

```
1  int main()
2  {
3      int N = 100;
4      int size = N * sizeof(float);
5
6      float* host_mem = (float*) malloc(size);
7      float* device_mem = NULL;
8      cudaMalloc(&device_mem, size);
9
10     for(int i = 0; i < N; i++)
11         host_mem[i] = i;
12
13     cudaMemcpy(device_mem, host_mem, size, cudaMemcpyHostToDevice);
14
15     //do computation with device_mem
16
17     cudaMemcpy(host_mem, device_mem, size, cudaMemcpyDeviceToHost);
18     for(int i = 0; i < N; i++)
19         printf("%f\n", host_mem[i]);
20
21     cudaFree(device_mem);
22     free(host_mem);
23 }
```

Listing 2: Device memory allocation and deallocation

On line 10 of Listing 2, `cudaMalloc` is used to obtain `size` bytes of storage on the device. After the initialization of host memory on lines 10—11, the entire vector is copied to device using `cudaMemcpy` on line 13. An argument `cudaMemcpyHostToDevice` is provided to specify the source and destination memory types. On line 17 `cudaMemcpy` is used again to copy device memory back to host. Note that all other source and destination memory type combinations in the form `cudaMemcpy[Source]To[Dest]` can be given. Finally, when the device memory is no longer needed, it is release with a call to `cudaFree` on line 21. Each of the functions `cudaMalloc`, `cudaFree` and `cudaMemcpy` return a standardized error type `cudaError_t` [Cor24] which was not checked for brevity.

4.2 Parallel algorithms

We outline all of the algorithms used for efficient implementation of our simulation. Specifically, we need to implement all operations used in Algorithms 2 and 3.

4.2.1 Parallel for

One of the most common operations is executing piece of code over all items of a collection in parallel. We call this parallel for.

In Listing 1, we already saw one way of achieving this effect. A single block was launched with number of threads equal to the number of items N. However, since all threads of a single block need to execute on a single SM, their number is limited by the capability of the hardware. Currently, the maximum of threads per block is 1024 for all CUDA devices. Because of this, the saxpy kernel will fail to launch for $N > 1024$. We solve this by introducing the so called *grid stride loop* into our kernel, which will allow us to efficiently execute our kernel for any number of items and any launch parameters. Grid stride loop is shown in Listing 3.

```
1  __global__ void saxpy(float* result, float a, const float* x, const float* y,
2                        int N)
3  {
4      for (int i = blockIdx.x*blockDim.x + threadIdx.x;
5          i < N;
6          i += blockDim.x*gridDim.x) {
7          result[i] = a*x[i] + y[i];
8      }
9  }
10
11 int main()
12 {
13     float* result = ...
14     float* x = ...
15     float* y = ...
16     float a = ...;
17     int N = 5000;
18
19     int blockSize = 256;
20     int gridSize = 2;
21     saxpy<<<gridSize, blockSize>>>(result, a, x, y, N);
22 }
```

Listing 3: Grid stride loop saxpy kernel

`blockIdx`, `blockDim` and `gridDim` in Listing 3 are all built-in variables of type `dim3` similar to `threadIdx`. `blockIdx` holds the index of the currently executing block, `blockDim` holds the block size (here `blockSize`) and `gridDim` the number of blocks (here `gridSize`). On line 3, we convert the provided variables into an index of the worked on item, on line 4, we check if this index is within the bounds of our arrays. This ensures the above code will work even for sizes not necessarily dividable by `blockSize`. On line 5, we step by the total number of threads in the loop, the so called grid stride. Finally, on line 6, we perform the computation. On line 20, we launch the kernel, providing the number of items `N`. We are free to choose any block size and grid size.

The grid stride loop results in the following iteration pattern: thread t_0 with `threadIdx` $= 0$, `blockIdx` $= 0$ iterates over indices 0, 512, 1024, 1536, ..., 4608. Thread t_1 with `threadIdx` $= 1$, `blockIdx` $= 0$ iterates over indices 1, 513, 1025, 1537 ...4609. Generally, thread t_i iterates indices $\{zBG + i, z = 0, \ldots, \lfloor \frac{N}{BG} \rfloor\}$ for all $i \in \hat{B}$ where N is the number of items, B is block size and G is grid size. This means that threads t_0, ..., t_{255} always execute on adjacent indices

resulting in *coalesced memory accesses*, which is beneficial to the hardware and conceptually simple.

Here we used a common value 256 for block size and arbitrary grid size of 2. Choosing kernel launch parameters is usually done with heuristics. To achieve optimal performance launch parameters grid search or compiler assisted kernel analysis can be used [Jes23]. However because threads are grouped into *warps*, block size should always be divisible by 32 (see 4.2.3). As there is practically no upper bound to the number of launched blocks, we usually choose grid size to be much larger than are given in Listing 3. This ensures the block scheduler has sufficient number of concurrently executing blocks to optimally utilize the hardware [Cor24]. For simplicity, we choose grid size such that all threads iterate in the grid size loop only once, that is $G = \lceil \frac{N}{B} \rceil$.

We can package the kernel and kernel launch code into a convenient function template, which we will specialize for all desired computations by providing a lambda function. The code is given in Listing 4.

```
1  template <typename Function>
2  __global__ void parallel_for_kernel(int from, int N, Function func)
3  {
4      for (int i = blockIdx.x*blockDim.x + threadIdx.x;
5          i < N;
6          i += blockDim.x*gridDim.x)
7          func(from + i);
8  }
9
10 template <typename Function>
11 void parallel_for(int from, int to, Function func)
12 {
13     int blockSize = 256;
14     int gridSize = (to - from + blockSize - 1) / blockSize; //divide round up
15     parallel_for_kernel<<<gridSize, blockSize>>>(from, to-from, func);
16 }
17
18 int main()
19 {
20     float* result = ...
21     float* x = ...
22     float* y = ...
23     float a = ...;
24     int N = 5000;
25
26     parallel_for(0, N, [=]__device__ __host__(int i){
27         result[i] = a*x[i] + y[i];
28     });
29 }
```

Listing 4: Template parallel for kernel

The kernel code on lines 1—8 of Listing 4, differs from the grid stride loop in Listing 3 only

in by invoking a general code passed in via the `func` function parameter in place of the previous computation. The launching code on lines 10—16 is using $B = 256$, calculating $G = \lceil \frac{N}{B} \rceil$ and passing the arguments into the kernel. Finally, lines 26—28 show calling of the `parallel_for` function template. A lambda is provided capturing local variable by value (`[=]`) accepting index `int i` and performing the computation. A `__device__ __host__` specifier is applied to the lambda. Similar to the already encountered `__global__` these specifiers determine how the function is to be called. `__device__` specifies that the function should only be callable from a device kernel. `__host__` specifies that the function is only callable from host. This is implicitly added to any function not marked by any of the other specifiers. `__device__ __host__` together specify that this function can be called from both host and device. Note that a different byte code is generated for the host and device versions of this function. To be able to place these specifiers on lambdas, the code needs to be compiled with `--extended-lambda` NVCC compiler flag.

In Listing 5, we present a two dimensional version of the saxpy code with grid stride loop 3. The transformation from one dimensional to two dimensional code is relatively straightforward and as such we will only present one dimensional variants from now on. We can also wrap this code in a generic wrapper similar to Listing 4 to obtain `parallel_for_2D`.

```
__global__ void saxpy2D(float* result, float a, const float* x, const float* y,
                        int N, int M)
{
    for (int i = blockIdx.y*blockDim.y + threadIdx.y;
        i < M; i += blockDim.y*gridDim.y)
        for (int j = blockIdx.x*blockDim.x + threadIdx.x;
            j < N; j += blockDim.x*gridDim.x)
        {
            int I = i*N + j;
            result[I] = a*x[I] + y[I];
        }
}

int main()
{
    float* result = ...
    float* x = ...
    float* y = ...
    float a = ...;
    int N = 1000;
    int M = 3000;

    dim3 blockDim(16, 16);
    dim3 gridDim(4, 4);
    saxpy2D<<<gridDim, blockDim>>>(result, a, x, y, N, M);
}
```

Listing 5: Two dimensional saxpy kernel

4.2.2 Parallel tiled for

The parallel for given in Listing 4 can be used for a majority of the needed operations, including a specialized version of matrix multiplication for matrices in the form (3.17) with a nonzero diagonal and four nonzero off diagonals. For now, let us consider a simpler case of a tridiagonal matrix $A \in \mathbb{R}^{N\times N}$ such that $\forall i \in \hat{N}$

$$A_{i,j} = \begin{cases} a \text{ when } i = j, \\ b \text{ when } i = j-1, \\ c \text{ when } i = j+1, \\ 0 \text{ otherwise} \end{cases} \tag{4.1}$$

for some constants a, b, $c \in \mathbb{R}$. Consider the multiplication of vector $\boldsymbol{x} \in \mathbb{R}^N$ by matrix A

$$(A\boldsymbol{x})_i = bx_{i-1} + ax_i + cx_{i+1} \tag{4.2}$$

$\forall i \in \{1, \ldots, N-1\}$. Using `parallel_for`, this can be turned into a CUDA function in Listing 6.

```
1 void tridiag_mul(float* result, float a, float b, float c, const float* x, int N)
2 {
3     parallel_for(1, N-1, [=]__device__ __host__(int i){
4         result[i] = b*x[i-1] + a*x[i] + c*x[i+1];
5     });
6 }
```

Listing 6: Naive tridiagonal matrix multiplication kernel

We will refer to device memory obtained by calling `cudaMalloc` or similar as *global memory*. The kernel in Listing 6 perform 3 global memory reads, three multiplies and two additions for each item. As the computation performed is fast in comparison to the global memory latency, the scheduler is not able schedule useful work while the threads are waiting for the memory reads to complete. Thus most of the kernel execution time is spend idly waiting for data. If we consider the case of matrix in the form (4.2.2) with 5 nonzero diagonals and matrix coefficients that vary by row, this effect is even more pronounced. 10 global memory reads are required per processed element.

We eliminate some of these reads if we consider that neighbouring output elements read mostly the same values. That is for example in the tridiagonal matrix multiplication (4.2.2) both output elements $(A\boldsymbol{x})i$,$(A\boldsymbol{x})_{i+1}$ read x_i and x_{i+1}. We utilize *shared memory*, which is shared between all threads of a single thread block, to efficiently broadcast values between neighbouring threads. All of the needed input values for the entire block are loaded into shared memory, then each thread executes its operation reading only from shared memory, which is expected to be much faster.

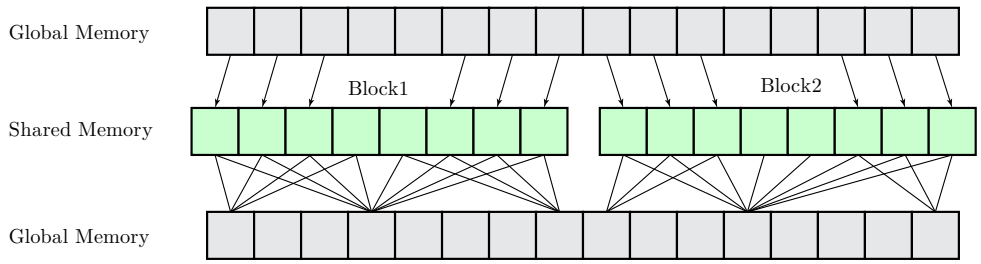


Figure 4.2: A graphic depicting usage of shared memory. Data is read from global memory (top) into per block shared memory (middle), computation is performed and results are stored back into global memory (bottom).

First, let us introduce programming with shared memory. A variable declared with memory space specifier `__shared__` is allocated in shared memory. Such a variable is accessible to all threads of a particular thread block. Because of this, any access to it needs to be handled with extra care to prevent data races. In the case of a single variable atomic operations need to be used. In case of an array, each thread of a thread block must write to a different index. When reading from shared memory which was updated by another thread, thread synchronization is required to ensure all threads have completed their update. This is done using the `__syncthreads()`

builtin function, which halts execution of the calling thread until all threads in its thread block reached this function call [Cor24].

In Listing 7 is an example kernel that sums "tiles" of 64 neighbouring items using shared memory. Notice that only one global memory read per output element is required instead of 64 if shared memory was not used.

```
1  __global__ void sum64(float* result, const float* x, int N)
2  {
3      assert(blockDim.x == 64);
4      __shared__ float shared[64];
5
6      int i = blockIdx.x*blockDim.x + threadIdx.x;
7      shared[threadIdx.x] = x[i];
8      __syncthreads();
9
10     float sum = 0;
11     for(int j = 0; j < 64; j++)
12         sum += shared[j];
13
14     result[i] = sum;
15 }
16
17 int main()
18 {
19     float* result = ...
20     float* x = ...
21     int N = 1024;
22
23     sum64<<<N/64, 64>>>(result, x, N);
24 }
```

Listing 7: Sum 64 items kernel

On line 4 of Listing 7, array in shared memory is declared. On line 6, an index into global memory is calculated and used to load values into shared memory on line 7. Notice that `threadIdx.x` is used to index into `shared` as shared memory is unique on per block basis. On line 8, `__syncthreads()` is called to ensure all threads have finished loading values into `shared` and thus these values are safe to read. On lines 10—14, sum of the items in the "tile" are calculated and output into `result`. The launch code on lines 17—25 remains in its usual form. Note that N needs to divisible by 64. Otherwise, the code will read outside of the provided arrays in global memory.

Next, we extend the code in Listing 7 above so that N can be of any value and the "tile size" (previously 64) can be set dynamically. For this, we need to obtain a dynamic amount of shared memory. This requires a new syntax and new launch parameter as shown in Listing 8.

```
1  __global__ void sum_tile(float* result, const float* x, int N)
2  {
3      extern __shared__ float shared[];
4      for (int i_base = blockIdx.x*blockDim.x; ; i_base += gridDim.x*blockDim.x)
5      {
6          if(i_base >= N)
7              break;
8
9          int i = i_base + threadIdx.x;
10         float read = 0;
11         if(i < N)
12             read = x[i];
13
14         shared[threadIdx.x] = read;
15         __syncthreads();
16
17         if(i < N)
18         {
19             float sum = 0;
20             for(int j = 0; j < blockDim.x; j++)
21                 sum += shared[j];
22
23             result[i] = sum;
24         }
25         __syncthreads();
26     }
27 }
28
29 int main()
30 {
31     float* result = ...
32     float* x = ...
33     int N = 5000;
34     int tileSize = 700;
35
36     int blockSize = tileSize;
37     int gridSize = (N + blockSize - 1)/blockSize;
38     int sharedMemSize = blockSize*sizeof(float);
39     sum_tile<<<gridSize, blockSize, sharedMemSize>>>(result, x, N);
40 }
```

Listing 8: Sum tile kernel for general value of N

On line 3 of Listing 8, we used a different syntax to obtain dynamically sized array in shared memory. On lines 4—7, we used grid stride loop to handle arbitrary sizes. All threads of a given thread block need to execute `__syncthreads()`, else a deadlock would occur. Because of this,

extra care needs to be taken around the stop condition, which needs to evaluate the same for all threads within a thread block. On lines 9—15, we load values into shared memory. This time, we only read values out of global memory if the thread's output index is inside the given array, else we load value 0. This ensures the whole `shared` array is initialized. Next, on lines 17—24, we perform the computation on threads whose output index is within the array. Then, on line 25, we synchronize one last time. This ensures no thread starts executing the next iteration overriding values in the `shared` array while computation is still in progress. Finally, on lines 36—39, we launch the kernel. `blockSize` is set to the size of the tile (max 1024) and `gridSize` is set according to $G = \lceil \frac{N}{B} \rceil$. The needed shared memory size per block `sharedMemSize` is calculated and given as a third launch parameter. This specifies the amount of memory to be allocated for the `shared` array.

Note that shared memory per block is a limited resource usually in tens of kilobytes on a typical CUDA enabled GPU. Devices of the NVIDIA Hopper architecture have up to 227 KB available shared memory per block [Cor24]. For the kernel in Listing 8 operating on `float`, supplying maximum block size $B = 1024$ results in 4KB shared memory used. While not a concern here, shared memory usage is another factor affecting possible range of launch parameters.

We are ready to present the kernel for the tridiagonal matrix multiplication in Listing 9. Because each thread with output index i reads values at indices $i - 1$, i, $i + 1$, we only execute the computation on threads in the interior of the thread block. More generally only for threads $t_r, \ldots, t_{B-r-1}$, where r is the "radius", ie., distance between the index i and furthest index read by the thread with output index i. Here $r = 1$.

```
1  __global__ void tridiag_mul_shared(float* result, float a, float b, float c,
2                                     const float* x, int N)
3  {
4      extern __shared__ float shared[];
5
6      int r = 1;
7      int ti = threadIdx.x;
8      for (int bi = blockIdx.x; ; bi += gridDim.x)
9      {
10         int i_base = bi*blockDim.x - 2*bi*r;
11         if(i_base >= N)
12             break;
13
14         int i = i_base - r + ti;
15         float val = 0;
16
17         if(0 <= i && i < N)
18             val = x[i];
19
20         shared[ti] = val;
21         __syncthreads();
22
23         if(r <= ti && ti < blockDim.x-r && i < N)
24             result[i] = b*shared[ti-1] + a*shared[ti] + c*shared[ti+1];
25
26         __syncthreads();
27     }
28 }
```

Listing 9: Tridiagonal matrix multiplication kernel

The code in Listing 9 can be extended to two dimensions similar to parallel for in Listing 4. By templating this function over type `T` of the `shared` array and replacing lines 13—15 and 22 by an invocation of a function passed to the kernel, we can generalize this algorithm to be used in all scenarios necessary by the implementation. The resulting function template signature is shown in Listing 10. `gather` loads or produces data stored into shared memory and `func` performs the computation utilizing shared memory. For full, implementation see the project repository.

```
template <int r, typename T, typename Function, typename Gather>
void tiled_for(int from, int to, Gather gather, Function func);

//Gather and Function are expected to have signatures (pseudocode):
T gather(int i, int N, int r);
void func(int i, int sharedIdx, int tileSize, cont T* shared);
```

Listing 10: Templated tiled for kernel declaration

4.2.3 Parallel reduction

Last algorithm needed for efficient implementation of our simulation is the parallel reduction. This operation consists in repeatedly applying a reduction operator v to array of values until only one value is left. More formally, let T be a set of possible values of some type. Let $v : T \times T \to T$ be an operator that is associative, commutative and has identity element e ie., $v(e, x) = x \forall x \in T$. Then reduction is a function $R : T^N \to T : \boldsymbol{x} \mapsto v(v(...v(v(e, x_1), x_2), ...), x_n)$, for some $N \in \mathbb{N}$. Some examples of common used reduction operators include $+$, min and max. We also use reduction to implement dot product and various vector norms.

Note that some implementations require v to be only commutative. However, we will need all three properties listed. For the sake of simplicity, we will be using the max reduction operator with identity value $-\infty$ in all further discussions.

A serial implementation of reduction using max reduction operator, is shown in Listing 11.

false

```
float reduce(const float* x, int N)
{
    float max = -INFINITY;
    for(int i = 0; i < N; i++)
        max = fmaxf(max, x[i]);

    return max;
}
```

Listing 11: Serial reduction

Of course, we want to perform this operation in parallel. The first key idea is "folding". In each iteration, we divide the remaining array in half and perform parallel reduction of the first and second half. This shrinks the remaining array to half of its original size (given that its size was even). We stop when the remaining size is just one item. This operation needs $\lceil \log_2(N) \rceil$ iterations each involving a separate kernel launch. The code is given in Listing 12.

```
float global_mem_reduce(float* temp1, float* temp2, const float* x, int N)
{
    cudaMemcpy(temp1, x, N*sizeof(float), cudaMemcpyDeviceToDevice);
    while(N > 1)
    {
        int half = (N + 1) / 2;
        parallel_for(0, half, [=]__device__ __host__ (int i){
            if(i + half < N)
                temp2[i] = fmaxf(temp1[i], temp1[i + half]);
            else
                temp2[i] = temp1[i];
        });

        std::swap(temp1, temp2);
        N = half;
    }

    float max = -INFINITY;
    cudaMemcpy(&max, temp1, sizeof(float), cudaMemcpyDeviceToHost);
    return max;
}
```

Listing 12: Reduction in global memory

We can optimize the number of kernel launches and accesses to global memory by utilizing shared memory. First, we perform partial reduction in global memory until only G items remain, where G is the grid size (see Section 4.2.1). Then a per block reduction is performed utilizing shared memory. The per block reduction result is written into a smaller output array. This yields a single value per block stored in global memory. The remaining G items are either reduced by invoking the kernel again or are reduced on the CPU if G is small. For problem sizes encountered by our simulation, this process yields one or two kernel launches depending on the choice of G. The resulting code is given in Listing 13. Additionally an optimization on the kernel launching code has been performed eliminating one `cudaMemcpy` call.

```
__global__ void shared_mem_reduce_kernel(float* output, const float* input, int N)
{
    assert((blockDim.x & (blockDim.x-1)) == 0 && "must be power of two!");
    extern __shared__ float shared[];

    int ti = threadIdx.x;
    float reduced = -INFINITY;
    for(int i = blockIdx.x*blockDim.x + ti;
        i < N; i += blockDim.x*gridDim.x)
        reduced = fmaxf(input[i], -INFINITY);

    shared[ti] = reduced;
    __syncthreads();

    for(int sharedSize = blockDim.x/2; sharedSize > 0; sharedSize /= 2) {
        if(ti < sharedSize)
            shared[ti] = fmaxf(shared[ti], shared[ti + sharedSize]);
        __syncthreads();
    }

    if(ti == 0)
        output[blockIdx.x] = shared[0];
}

float shared_mem_reduce(float* temp1, float* temp2, const float* input, int N)
{
    const float* from = input;
    enum {MIN_SIZE = 64};
    while(N > MIN_SIZE) {
        int blockSize = 1024;
        int sharedMemSize = blockSize*sizeof(float);
        int gridSize = (N + blockSize - 1) / blockSize;

        shared_mem_reduce_kernel
            <<<gridSize, blockSize, sharedMemSize>>>(temp2, from, N);

        std::swap(temp1, temp2);
        N = gridSize;
        from = temp1;
    }
    float cpu[MIN_SIZE] = {0};
    cudaMemcpy(cpu, from, N*sizeof(float), cudaMemcpyDeviceToHost);

    float max = -INFINITY;
    for(int i = 0; i < N; i++)
        max = fmaxf(max, cpu[i]);
    return max;
}
```

Listing 13: Reduction in shared memory

We can optimize this implementation in Listing 13 further by utilizing *warps* which are collections of 32 threads. The CUDA processor creates, executes and schedules threads only on per warp basis. Prior to the NVIDIA Volta architecture, all threads in a single warp shared the same program counter. That is, all threads of a warp executed the same instructions similarly to SIMD (Single Instruction Multiple Data) programming. This also meant that the CUDA processors were unable to execute conditional `if`/`else` branches unless the condition evaluated the same for all threads in a warp. In those cases, the processor would execute both the `if` and `else` code block for all threads in a warp, but only store the computation result for the *active threads*, that is threads, for which the condition evaluated according to the currently executed branch. This process is called *warp divergence* and is undesired because of its sub-optimal performance. NVIDIA Volta architectures and later eliminate warp divergence by providing full concurrency between threads independent of warp. Each thread has its own program counter and is able to be scheduled independently. Still, warps play an important role as they allow efficient thread communication within a single warp [Cor24].

We use warp communication to share partial reduction results similarly to shared memory in Listing 13. First, partial reduction in global memory is performed until only B items per block remain. Then a warp level reduction is performed, yielding a value which is stored into shared memory and all threads of a block are synchronized. The first $B/32$ threads of a block load values from shared memory and perform warp level reduction again. This time the resulting $B/32^2$ values are stored into global output memory. With $B \leq 1024$ this yields a single value per block stored in global memory. Similarly to reduction in shared memory, the partial reduction results are reduced by a repeated kernel launch or are processed on the CPU. The resulting code is given in Listing 14.

```
__global__ void warp_reduce_kernel(float* output, const float* x, int N)
{
    enum {WARP_SIZE = 32};
    assert(blockDim.x > WARP_SIZE && blockDim.x % WARP_SIZE == 0);

    extern __shared__ float shared[];
    uint shared_size = blockDim.x / WARP_SIZE;

    float reduced = 0;
    for(int i = blockIdx.x*blockDim.x + threadIdx.x; i < N; i += blockDim.x*gridDim.x)
        reduced = fmaxf(reduced, x[i]);

    for (int offset = WARP_SIZE / 2; offset > 0; offset /= 2)
        reduced = fmaxf(__shfl_down_sync(0xffffffffU, reduced, offset), reduced);

    uint ti = threadIdx.x;
    if (ti % WARP_SIZE == 0)
        shared[ti / WARP_SIZE] = reduced;

    __syncthreads();
    uint ballot_mask = __ballot_sync(0xffffffffU, ti < shared_size);
    if (ti < shared_size)
    {
        reduced = shared[ti];
        for (int offset = WARP_SIZE / 2; offset > 0; offset /= 2)
            reduced = fmaxf(__shfl_down_sync(ballot_mask, reduced, offset), reduced);
    }

    if (ti == 0)
        output[blockIdx.x] = reduced;
}

//same as shared_mem_reduce but calls warp_reduce_kernel
float warp_reduce(float* temp1, float* temp2, const float* input, int N);
```

Listing 14: Reduction using warp instructions

Like in the parallel algorithms already discussed, we template the reduction kernel in Listing 14 over the specific type and reduction operator used. Additionally we provide "producer" lambda which abstracts the retrieval of reduced values. This creates the so called map-reduce operation, which can be used to efficiently implement many vector operations such as the Euclid norm shown is Listing 15. Note that the implementation requires both reduction function and its identity element which are provided via the functor type `Reduction`. While custom reduction operators can be defined, the common operators are provided. This allows the implementation to use warp level reduction intrinsics where eligible. These intrinsics replace the lines 13—14

and 25—26 in Listing 14 and are expected to be much faster. An example of such intrinsic for the `int` type and add reduction operator is `__reduce_add_sync` [Cor24].

```
template <typename T, class Reduction, typename Producer>
T produce_reduce(int N, Producer produce, int cpu_reduce = 256);

float euclid_norm(const float *a, int N)
{
    float output = produce_reduce<float, Reduce::Add>(N,
        [=]__host__ __device__(int i){
            return a[i]*a[i];
        });
    return sqrtf(output);
}
```

Listing 15: Template reduction

4.3 Algorithm benchmarks

We try to measure the speed of the proposed algorithms in Section 4.2. Due to time constraints, only the parallel reduction algorithm given in Section 4.2.3 is measured.

The benchmark procedure consists of the following steps: firstly, an array of size $N \in \mathbb{N}$ filled with pseudo-random values is generated. This array is used as the input to the benchmarked algorithm. Then, the algorithm is run repeatedly for t_{warmup} seconds. This ensures all relevant data is cached. Next, the algorithm is run for t_{measure} seconds, measuring and storing its execution time. Additionally, if more then N_{samples} running times are collected, the benchmark is stopped. Finally, the running process is put to sleep for t_{cooldown}, ensuring no heat throttling occurs. We report the median running time of the samples collected.

We benchmarked the parallel reduction algorithm given in Section 4.2.3, CUDA Thrust library `thrust::reduce` and naive serial reduction given in Listing 11. The algorithms use the max reduction operator on the `float` data type.

The benchmarking procedure was run repeatedly for multiple input sizes $N \in \{256^2, 512^2, 1024^2, 2048^2, 4096^2, 2 \times 4096^2\}$ and parameters $t_{\text{warmup}} = 0.3$, $t_{\text{measure}} = 3$, $t_{\text{cooldown}} = 5$, $N_{\text{samples}} = 1024^2$.

The benchmark was run on a laptop using Ubuntu 22.04.4 LTS, NVIDIA GeForce MX450, Driver Version: 535.183.01, CUDA Version: 12.2, 11th Gen Intel(R) Core(TM) i5-1135G7 @ 2.40GHz.

It is worth noting that the benchmarking procedure never collected more than N_{samples} thus the benchmark run for the full t_{measure} duration. The results are summarized in Table 4.1 and Figure 4.3. The full benchmarking code implementation can be found in the project repository.

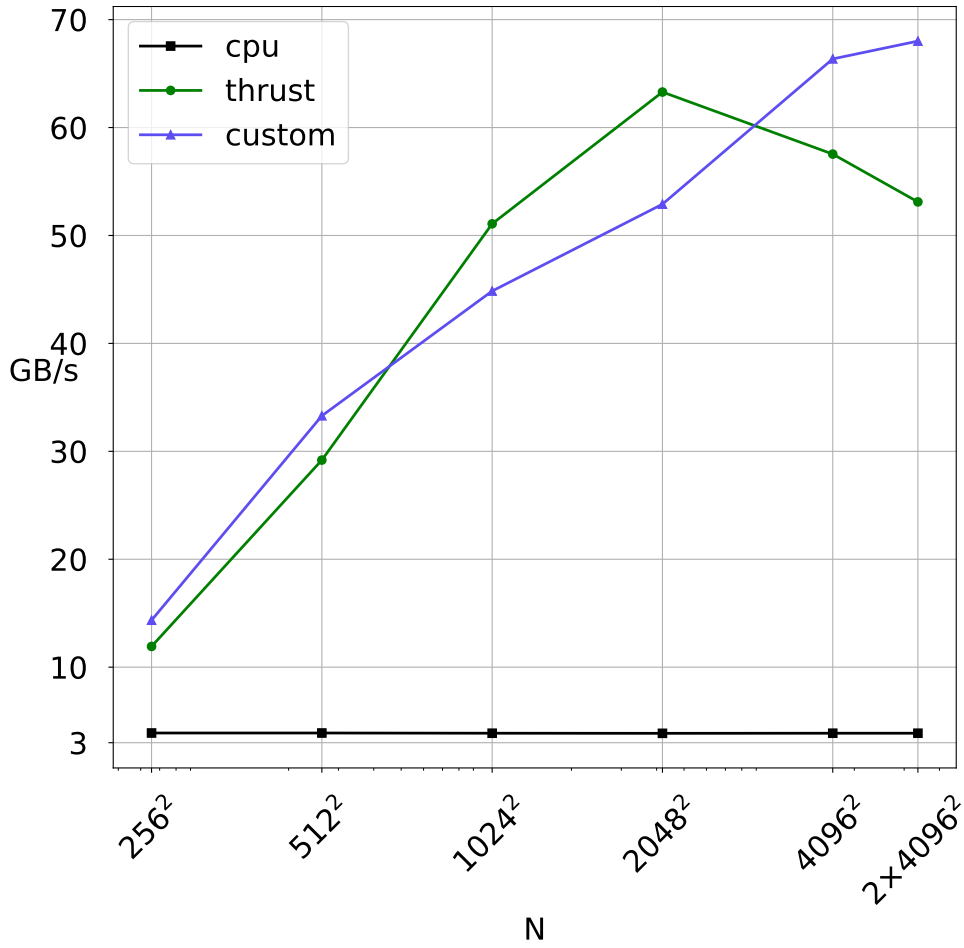


Figure 4.3: Median running times of the benchmarked reduction algorithms expressed in gigabytes of processed data per second. On the x logarithmic axis is the size of the dataset N.

N	256^2	512^2	1024^2	2048^2	4096^2	2×4096^2
cpu	6.26×10^{-5}	2.50×10^{-4}	1.01×10^{-3}	4.04×10^{-3}	1.61×10^{-2}	3.22×10^{-2}
thrust	2.05×10^{-5}	3.34×10^{-5}	7.65×10^{-5}	2.47×10^{-4}	1.09×10^{-3}	2.35×10^{-3}
custom	1.70×10^{-5}	2.93×10^{-5}	8.71×10^{-5}	2.95×10^{-4}	9.42×10^{-4}	1.84×10^{-3}

Table 4.1: Median running times in seconds of the benchmarked reduction algorithms.

From the collected results in Table 4.1 and Figure 4.3, we conclude that the parallel reduction algorithm given in Section 4.2.3, outperforms the implementation in the Thrust library for small and large input sizes sizes on the used hardware. However, for input sizes of $N = 1024^2, 2048^2$ the Thrust implementation achieves considerably shorter running times.

Chapter 5

Results

We present and discuss the results obtained by running the algorithm developed by using the techniques in Chapter 3 in various settings. First, we explore the impact of different simulation parameters on resulting crystal structures. Then, comparison between different integration methods is performed. Finally, the effect of repeated time stepping and correction term is quantified (see Sections 3.3.1 and 3.3.2).

5.1 Parameter setting

5.1.1 Boundary conditions

For the temperature field T the homogeneous Dirichlet boundary condition (see Section 2.2.3.1) written as

$$T|_{\partial\Omega} = 0 \text{ on } \mathcal{T} \times \partial\Omega, \tag{5.1}$$

which can be interpreted a controlled cooling so that $\partial\Omega$ is kept at constant temperature. The temperature in the domain Ω approaches 0. This results in unbounded growth of the crystal which eventually fills the whole domain Ω. This process is shown in Figure 5.6.

The homogeneous Neumann boundary condition (see Section 2.2.3.2) written as

$$\nabla T \cdot \boldsymbol{n} = 0 \text{ on } \mathcal{T} \times \partial\Omega, \tag{5.2}$$

means the system is adiabatically isolated. No heat is transferred across the boundary $\partial\Omega$. As the system approaches thermodynamic equilibrium, it minimizes its interfacial energy, making the crystal structure more rounded in a process called *coarsening*. The crystal in limit approaches the so called *Wulff shape* [Str12], [Gur93]. This process is shown in Figure 5.2.

For the phase filed Φ, the homogeneous Dirichlet boundary condition

$$\Phi|_{\partial\Omega} = 0 \text{ on } \mathcal{T} \times \partial\Omega \tag{5.3}$$

represents an unrealistic jump change from the crystal structure in the domain Ω to perfect liquid phase on the boundary $\partial\Omega$. This creates high phase field flux for crystal structures near the boundary, which through the latent heat L term rapidly cools the surroundings, prompting spurious crystal growth. On the other hand, for Φ the homogeneous Neumann boundary condition

$$\nabla\Phi \cdot \boldsymbol{n} = 0 \text{ on } \mathcal{T} \times \partial\Omega \tag{5.4}$$

allows the crystal to reach the boundary but does not cause spurious crystal growth. Because of this the homogeneous Neumann boundary condition is used in all subsequent simulations.

5.1.2 Initial conditions

Initial conditions are specified to conform to the modeled process of solidification in an undercooled medium. The temperature is uniformly initialized below the melting point

$$T_{ini}(\boldsymbol{x}) = 0 \qquad \forall \boldsymbol{x} \in \Omega \tag{5.5}$$

and the phase is initialized to liquid phase except for initial nucleus of solid phase

$$\Phi_{ini}(\boldsymbol{x}) = \begin{cases} 1 & \text{when} \quad d(\boldsymbol{x}) < R_{ini} - \xi_0/2, \\ 1 - \dfrac{d(\boldsymbol{x}) - R_{ini} + \xi_0/2}{\xi_0} & \text{when} \quad R_{ini} + \xi_0/2 < d(\boldsymbol{x}) < R_{ini} - \xi_0/2, \\ 0 & \text{when} \quad R_{ini} + \xi_0/2 \leq d(\boldsymbol{x}), \end{cases} \quad \forall \boldsymbol{x} \in \Omega, \tag{5.6}$$

where $d(\boldsymbol{x}) = |\boldsymbol{x} - \boldsymbol{x}_0|$ is the distance to the nucleus center $\boldsymbol{x}_0$ and $R_{ini} \in \mathbb{R}$ is the nucleus radius [Str12]. $\xi_0 \in \mathbb{R}$ determines the size of linear transition between the solid and liquid phase. The value of ξ_0 is chosen so that the initial condition better approximates the shape of the phase interface (see Figure 5.7). While even with $\xi_0 = 0$ the interface quickly develops to the expected shape, properly chosen ξ_0 helps to reduce sudden changes in temperature and phase fields near the start of the simulation, which could cause the employed schemes to diverge. This effect can be observed in Figure 5.10.

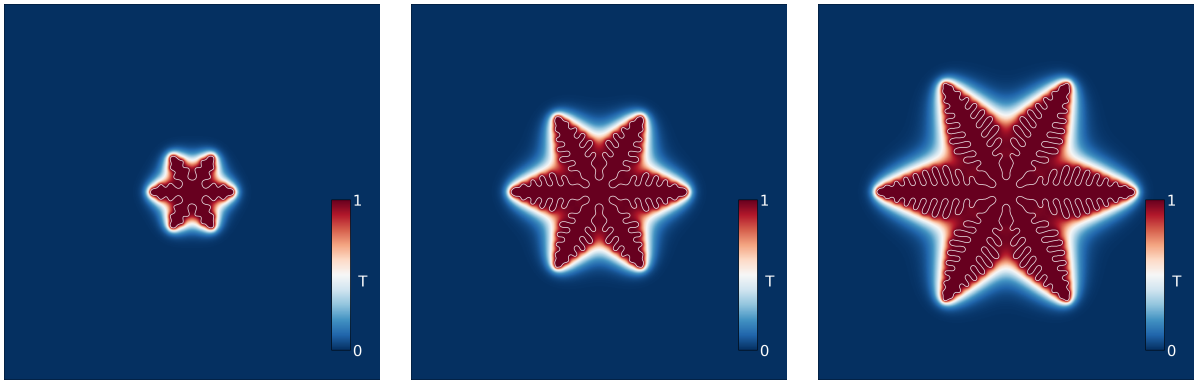
5.1.3 Simulation setup

Homogeneous Neumann boundary conditions are used for both the temperature and the phase field. Initial conditions given by (5.4) and (5.6) are used with $\boldsymbol{x}_0 = (2, 2)$, $R_{ini} = 0.04$, $\xi_0 = 8\xi$. The model parameters are summarized in the Table 5.1.

For the RKM scheme given in Section 3.1, tolerance $\varepsilon = 5 \times 10^{-3}$ is used. For the S-I scheme given in Section 3.2, coefficient $\gamma = 1$ and tolerance $\varepsilon = 5 \times 10^{-9}$ are used.

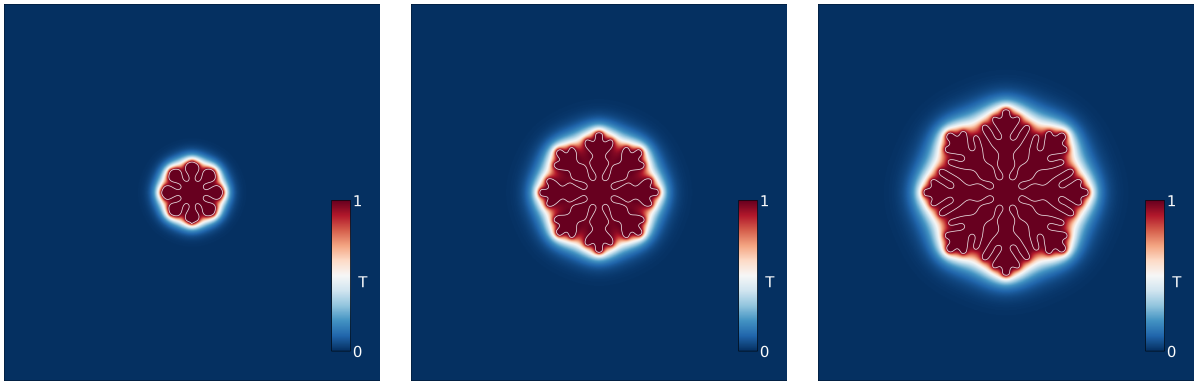
$\Omega = (0, 4) \times (0, 4)$	$\mathcal{T} = [0, 0.04]$
$n_x = n_y = 1024$	$\Delta t = 5 \times 10^{-5}$
$\Delta x = \Delta y = 4/1024$	$\xi = 0.0043$
$a = 2$	$\alpha = 3$
$b = 1$	$\beta = 1400$
$L = 2$	$T^* = 1$
$S = 0.3$	$m = 6$
$\theta_0 = 0$	

Table 5.1: Simulation model parameters



$t = 0.01$ $t = 0.0275$ $t = 0.04$

Figure 5.1: Evolution of the temperature field and the phase interface Γ (in white) over the time interval $\mathcal{T} = [0, 0.04]$ using the S-I scheme. Simulation setup given in Section 5.1.3 was used. The simulation running on hardware given in Section 4.3 run for 354 seconds, with average time of 44 milliseconds per iteration. The conjugate gradient method used in the S-I scheme performed on 4 iterations to converge for the phase field and 6 iterations to converge for the temperature field. Note that the performance characteristics of the S-I scheme are hard to generalize as the convergence and thus runtime is partly determined by the choice of model parameters.



$t = 0.01$ $t = 0.032$ $t = 0.06$

Figure 5.2: Evolution of the temperature field and the phase interface Γ (in white) over the time interval $\mathcal{T} = [0, 0.06]$ using the S-I scheme. Simulation setup given in Section 5.1.3 was used. Anisotropy has been changed to 8-fold.

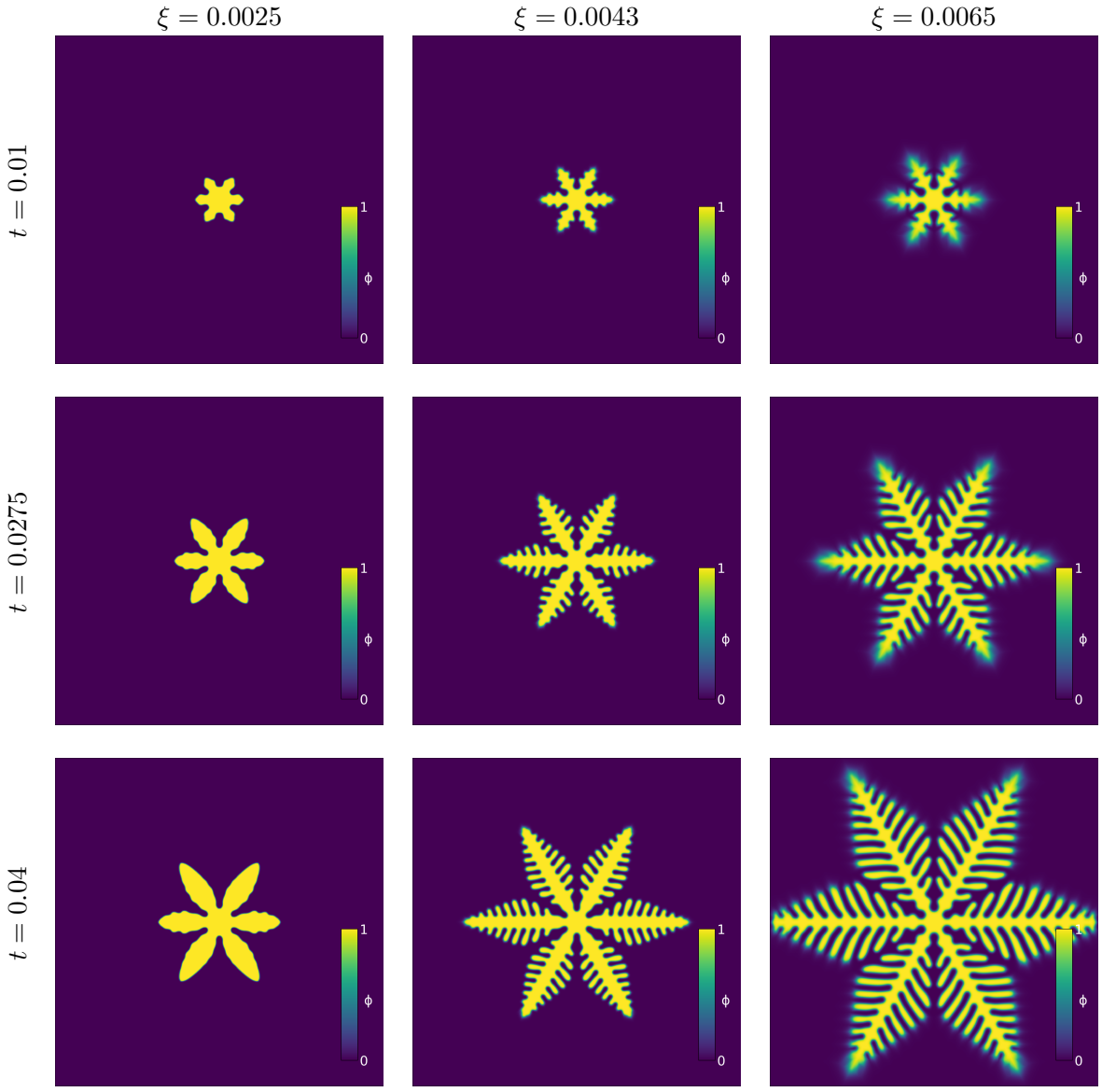


Figure 5.3: Comparison of phase fields for different values of parameter ξ using the S-I scheme. Simulation setup given in Section 5.1.3 was used. Parameter ξ was taken as one of $\{0.0025, 0.0043, 0.0065\}$. The width of the *phase interface*, that is the region where the phase field Φ is between 0 and 1, is dependent on the value of ξ. For high value of $\xi = 0.0065$ the resulting phase field interface is thicker and heavily distorted in the direction of crystal growth. For medium values of $\xi = 0.0043$, roughly around the values of $\Delta x, \Delta y$, the phase field interface has the same thickness around the whole crystal. For small values of $\xi = 0.0025$, the proper shape of the phase interface cannot be accurately represented on the mesh, reducing the rate of crystal growth.

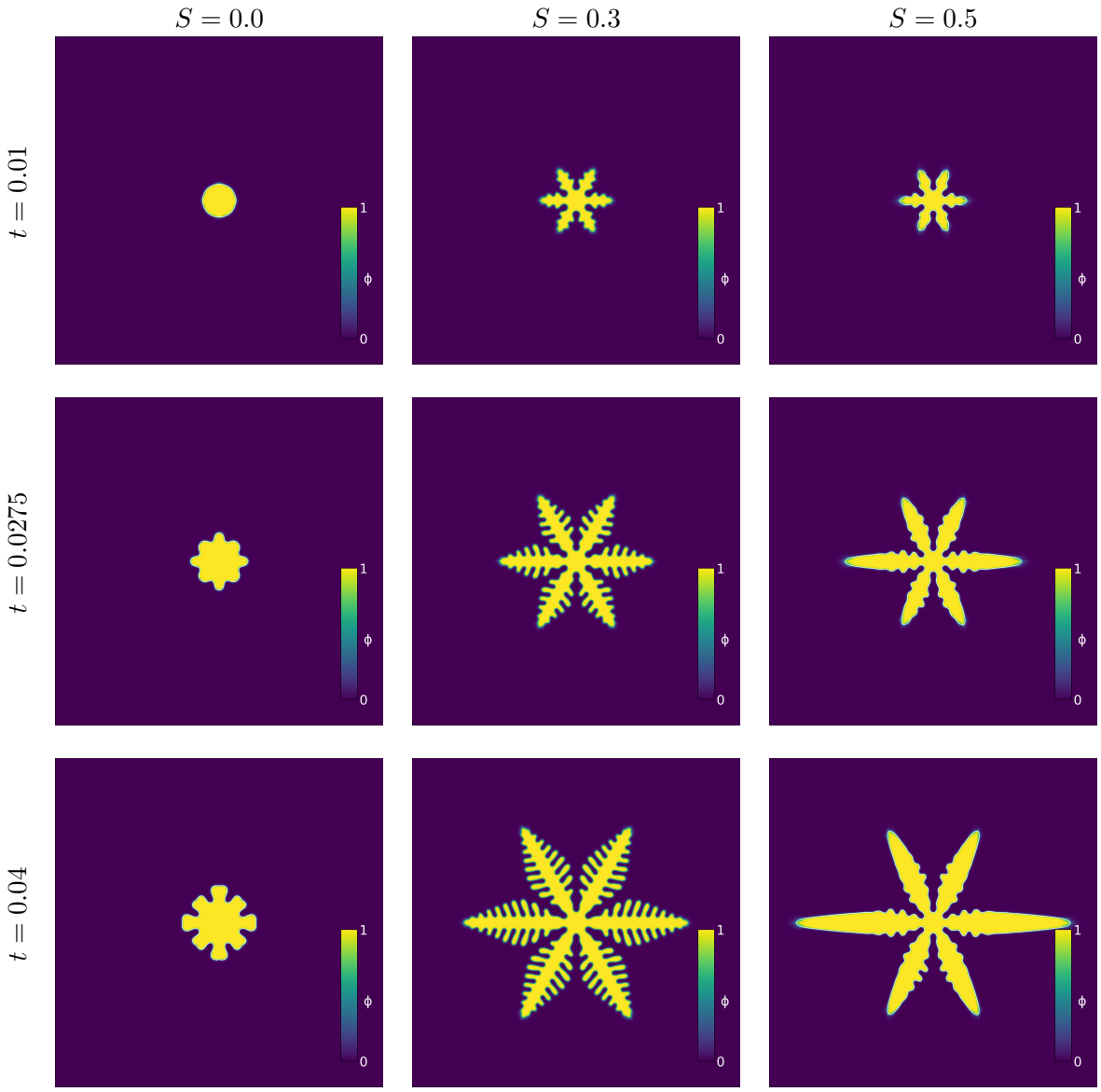
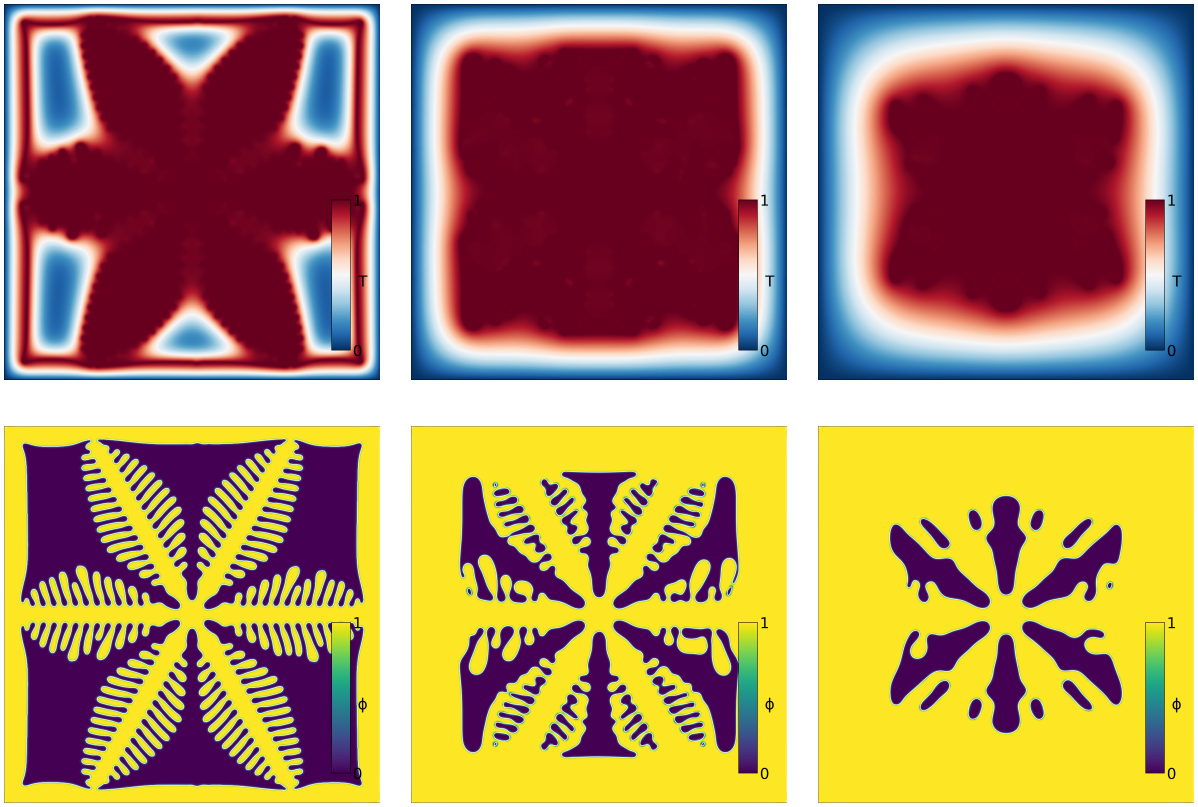
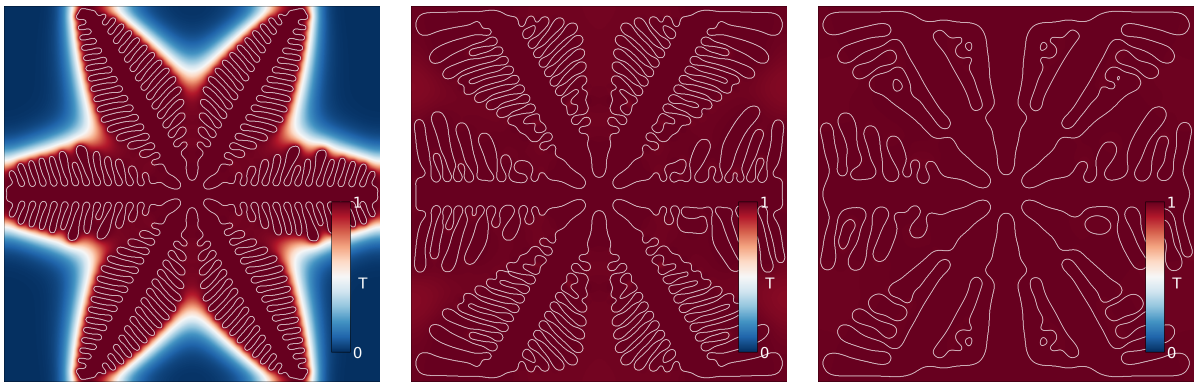


Figure 5.4: Comparison of phase fields for different values of anisotropy strength S using the S-I scheme. Simulation setup given in Section 5.1.3 was used. S was taken as one of $\{0, 0.3, 0.5\}$. For low and high values of anisotropy strength, the underlying regular grid has a visible impact on the resulting crystal structure. Specifically in the case of $S = 0$, the crystal should remain a perfect ball.



$t = 0.1$ $t = 0.3$ $t = 0.6$

Figure 5.5: Evolution of the temperature field T (top) and the phase field Φ (bottom) over the time interval $\mathcal{T} = [0, 0.6]$ using the S-I scheme. Simulation setup given in Section 5.1.3 alongside Dirichlet boundary conditions for the the temperature field (5.1) were used. Due to the controlled cooling at the boundary, the crystal expands, eventually filling the entire domain Ω.



$t = 0.1$ $t = 0.3$ $t = 0.6$

Figure 5.6: Evolution of the temperature field and the phase interface Γ (in white) over the time interval $\mathcal{T} = [0, 0.6]$. Simulation setup given in Section 5.1.3 was used. The system reaches state of near thermodynamic equilibrium and coarsening of the crystal structure is observed (see Section 5.1.1).

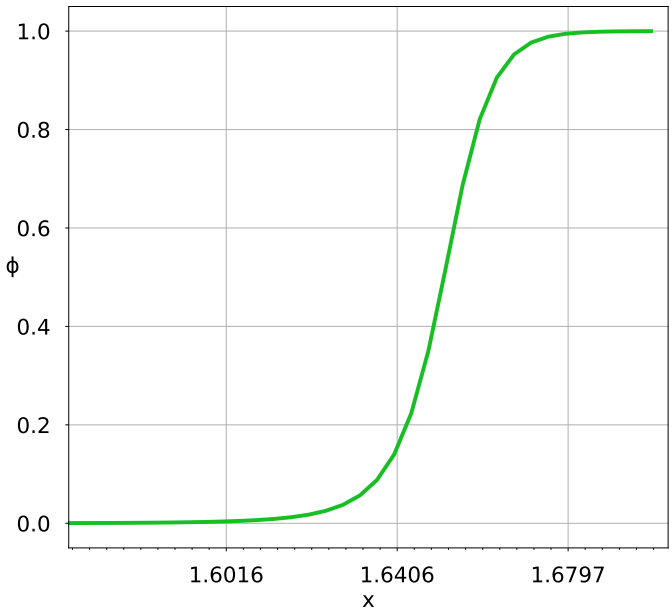


Figure 5.7: Cross section of phase field at $y = 0$ showing the shape of the phase interface. Simulation setup given in Section 5.1.3 was used. Anisotropy strength S was set to 0. Small ticks on the x axis are Δx apart. The phase interface is approximately $20\Delta x$ or 18ξ thick, where we consider the phase interface to be all values of Φ between $[0.01, 0.99]$. Similar values of ξ have been observed in [SWB21].

5.2 Model comparison

We verify the consistency of the proposed numerical methods by cross-comparison. We show results of the semi-implcit and various explicit schemes and extensively quantify the difference

between these results.

5.2.1 Phase field difference metric

To asses similarity of the resulting crystal structures, we will introduce two metrics. Let $\Phi_1, \Phi_2 : \mathcal{T} \times \Omega \to [0,1]$ be phase fields as defined in Section 1.2 and let $\Phi_{1,d}, \Phi_{2,d} : \mathcal{T} \times \mathcal{M} \to [0,1]$ be their corresponding mesh functions as described in Section 2.1. For the purpose of this section we are only interested in their values at some concrete time $t \in \mathcal{T}$, so let $\Phi_i = \Phi_i(t, \cdot)$ and $\Phi_{i,d} = \Phi_{i,d}(t, \cdot)$, $i = 1, 2$. Lastly, we employ an additional discretization function $D : \mathbb{R} \to \{0, 1\}$ which is defined as

$$D(x) = \begin{cases} 1, & \text{when } x \geq \frac{1}{2} \\ 0 & \text{otherwise} \end{cases} \qquad \forall x \in \mathbb{R}. \tag{5.7}$$

We define the phase *phase field norm* (PFN)

$$PFN(\Phi_1) = \|\Phi_1\|_{L_1(\Omega)} = \int_\Omega |\Phi_1(\boldsymbol{x})| \mathrm{d}\boldsymbol{x}, \tag{5.8}$$

the *phase field distance* (PFD) as

$$PFD(\Phi_1, \Phi_2) = PFN(\Phi_1 - \Phi_2), \tag{5.9}$$

and the *phase field discrete distance* (PFDD) as

$$PFDD(\Phi_1, \Phi_2) = PFN(D \circ \Phi_1 - D \circ \Phi_2). \tag{5.10}$$

The phase field discrete distance (PFDD) (5.10) represents the volume of the symmetric set difference of the two regions occupied by the solid phase. That is

$$PFDD(\Phi_1, \Phi_2) = m\left(\left\{\boldsymbol{x} \in \Omega \;\middle|\; \Phi_1(x) \geq \frac{1}{2}\right\} \triangle \left\{\boldsymbol{x} \in \Omega \;\middle|\; \Phi_2(x) \geq \frac{1}{2}\right\}\right), \tag{5.11}$$

where m is the Lebesgue measure and $\triangle$ is the symmetric set difference defined for two sets A, B as

$$A \triangle B = (A \setminus B) \cup (B \setminus A). \tag{5.12}$$

However in certain cases, such as when comparing the value of the step residual r_Φ, we are interested in small difference of the phase field better captured by the phase field distance (PFD) (5.9). Because of the continuous nature of the model, these small differences are enough to, for example, indicate where the crystal might branch off in the future. Note that in particular L_∞ distance is an inconvenient tool for assessing the differences for the phase field as even small differences in the crystal position result in the maximum difference close to 1.

To extend the above definitions for the mesh functions $\Phi_{1,d}, \Phi_{2,d}$ we introduce the piecewise constant interpolation function

$$I : \Omega \to \mathcal{M} : \boldsymbol{x} \mapsto C \in \mathcal{M} \text{ such that } \boldsymbol{x} \in C, \tag{5.13}$$

and use it to define

$$PFN(\Phi_d) = PFN(\Phi_d \circ I), \tag{5.14}$$
$$PFD(\Phi_{1,d}, \Phi_{2,d}) = PFD(\Phi_{1,d} \circ I, \Phi_{2,d} \circ I), \tag{5.15}$$
$$PFDD(\Phi_{1,d}, \Phi_{2,d}) = PFDD(\Phi_{1,d} \circ I, \Phi_{2,d} \circ I). \tag{5.16}$$

Specifically for uniform rectangular mesh, we can compute PFDD as

$$PFDD(\Phi_{1,d}, \Phi_{2,d}) = \Delta x \Delta y \|D(\boldsymbol{F}_1) - D(\boldsymbol{F}_2)\|_1, \tag{5.17}$$

where $\boldsymbol{F}_1, \boldsymbol{F}_2 \in \mathbb{R}^N$ are vectors of the mesh functions $\Phi_{1,d}, \Phi_{2,d}$, respectively, given by (3.12) and function D is applied element-wise. PFN and PFD can be computed similarly.

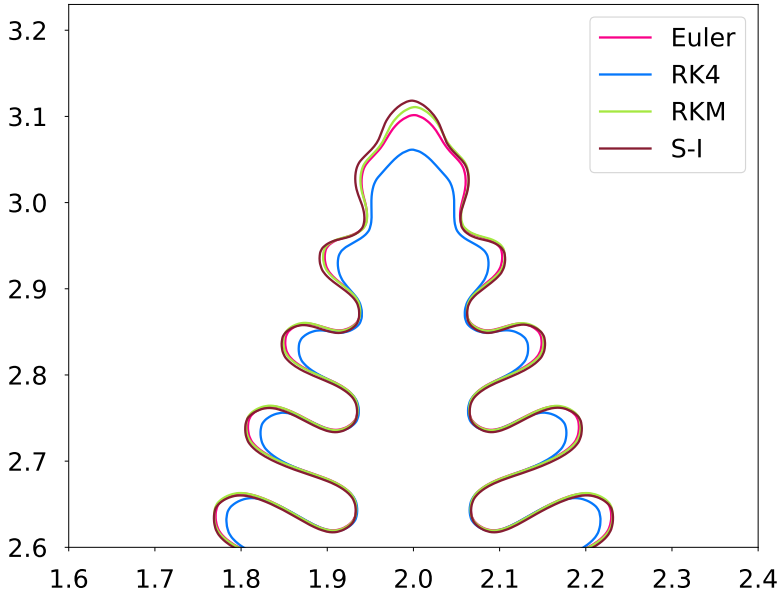


Figure 5.8: Comparison of crystal structures for different numerical schemes at $t = 0.04$. Simulation setup given in Section 5.1.3 was used with changed $\Delta t = 2.5 \times 10^{-5}$. The phase interface obtained using the Euler, RK4, RKM and S-I schemes is shown.

Schemes	Euler	RK4	RKM	S-I
Euler	0	9.02×10^{-2}	1.52×10^{-2}	5.36×10^{-2}
RK4	9.02×10^{-2}	0	1.01×10^{-1}	1.24×10^{-1}
RKM	1.52×10^{-2}	1.01×10^{-1}	0	4.43×10^{-2}
S-I	5.36×10^{-2}	1.24×10^{-1}	4.43×10^{-2}	0

Table 5.2: Phase field discrete distance (PFDD) between results obtained by different integration schemes. Refer to Figure 5.8 for the experiment setup.

Schemes	Euler	RK4	RKM	S-I
Euler	0	9.05×10^{-2}	1.49×10^{-2}	5.20×10^{-2}
RK4	9.05×10^{-2}	0	1.01×10^{-1}	1.23×10^{-1}
RKM	1.49×10^{-2}	1.01×10^{-1}	0	4.32×10^{-2}
S-I	5.20×10^{-2}	1.23×10^{-1}	4.32×10^{-2}	0

Table 5.3: Phase field distance (PFD) between results obtained by different integration schemes. Refer to Figure 5.8 for the experiment setup.

From Figure 5.8 and Tables 5.3, 5.2, we conclude that the proposed schemes produce similar crystal structure, with the RK scheme differing most significantly. Because of this, it will not be included in further discussions. Furthermore, the values obtained from phase field distance (PFD) and phase field discrete distance (PFDD) are almost identical. This is caused by the phase field function, which is, apart from the thin phase interface, very close to either 0 or 1. As such we will only list the PFDD distance in scheme comparison from now on.

5.3 Correction impact

We explore the effect of the proposed solutions to reduce the error introduced by operator splitting, namely repeated time stepping and correction term (see Sections 3.3.1 and 3.3.2).

Even though repeated time stepping is applicable to all proposed numerical schemes, it has only been developed for the the S-I and Euler schemes, which will be used in the presented results. The correction term modification is presented for the S-I, RKM and Euler schemes.

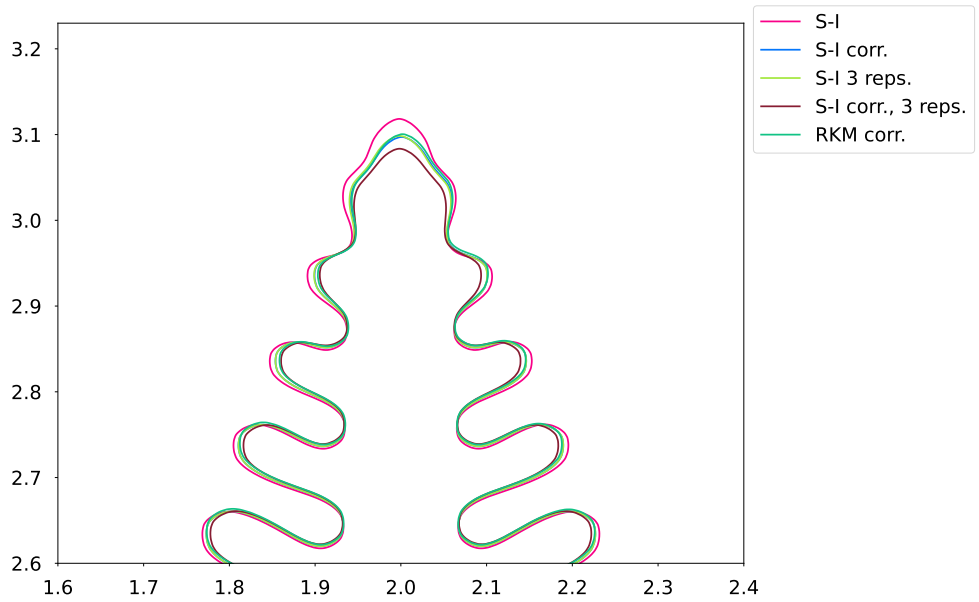
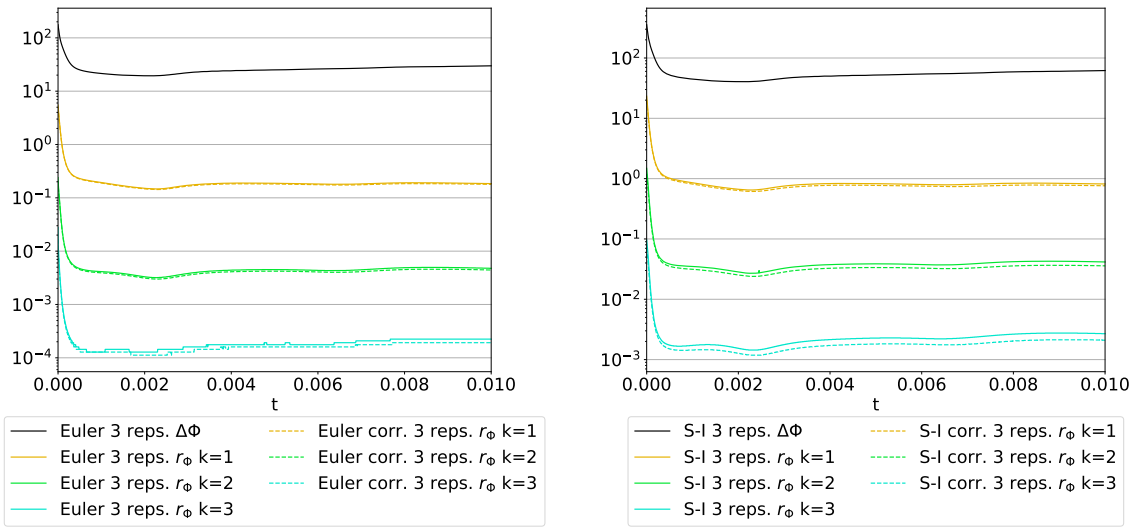


Figure 5.9: Comparison of crystal structures for different numerical schemes with repeated time stepping and correction term modifications at $t = 0.04$. Simulation setup given in Section 5.1.3 was used except for the setting $\Delta t = 2.5 \times 10^{-5}$. Schemes marked with the abbreviation "corr." use the correction term modification, described in Section 3.3.2. Schemes marked with the abbreviation "3 reps." use repeated time stepping, described in Section 3.3.1, with $M = 3$, $\varepsilon = 0$, $\alpha = 1$ (see Algorithm 4).

As shown in Figure 5.9, the crystal structure is very similar for all modifications. Operator splitting corrections make the crystal grow "slower", so the majority of the error is found at the tips of a dendrites where the crystal grows the fastest. When used separately, repeated time stepping and correction term modification both achieve remarkably similar results for the S-I scheme (see Table 5.4). When used in conjunction, they seem to produce different structures compared to the rest of the schemes.



(a) Explicit scheme using Euler method

(b) Semi-Implicit scheme

Figure 5.10: Evolution of the phase field norm (PFN) of the n-th step residual $\boldsymbol{r}_\Phi$ for the Euler and S-I schemes. $\Delta\Phi$ is the phase field distance (PFD) between the current and next time step, that is $\Delta\Phi = PFD(\boldsymbol{F}^{n+1}, \boldsymbol{F}^n)$, where $\boldsymbol{F}^n$ represents discretized phase field Φ is in the timestep n given by (3.12). r_Φ $\#k$, $k \in \{1, 2, 3\}$ marks the value of $PFN(r_\Phi)$ in the k-th iteration of Algorithm 4. Schemes marked with the abbreviation "corr." use the correction term modification, described in Section 3.3.2. Schemes marked with the abbreviation "3 reps." use repeated time stepping, described in Section 3.3.1, with $M = 3$, $\varepsilon = 0$, $\alpha = 1$. Simulation setup given in Section 5.1.3 was used with changed $\Delta t = 2.5 \times 10^{-5}$ for the S-I scheme and $\Delta t = 1.2 \times 10^{-5}$ for the Euler scheme. High values near the start of the simulation are caused by the mismatch between the initial conditions given in Section 5.1.2 and the stable shape of the phase interface depicted in Figure 5.7. Discrete jumps in values close to 10^{-4} are caused by the inability of the used storage format to represent small numbers.

From Figure 5.10 we conclude that Algorithm 4 reduces the magnitude of the step residual $PFN(\boldsymbol{r}_\Phi)$ by more than an order of magnitude per iteration. On the other hand, the corrected schemes proposed in Section 3.3.2 have negligible impact on the value of $PFN(\boldsymbol{r}_\Phi)$. Similar trends are observed for $\boldsymbol{r}_\Phi$ in the L_2 and L_∞ norms.

Schemes	S-I	S-I corr.	S-I 3 reps.	S-I corr., 3 reps.	RKM corr.
S-I	0	5.52×10^{-2}	5.52×10^{-2}	1.05×10^{-1}	9.14×10^{-2}
S-I corr.	5.52×10^{-2}	0	1.56×10^{-3}	5.06×10^{-2}	4.01×10^{-2}
S-I 3 reps.	5.52×10^{-2}	1.56×10^{-3}	0	5.06×10^{-2}	4.02×10^{-2}
S-I corr., 3 reps.	1.05×10^{-1}	5.06×10^{-2}	5.06×10^{-2}	0	3.11×10^{-2}
RKM corr.	9.14×10^{-2}	4.01×10^{-2}	4.02×10^{-2}	3.11×10^{-2}	0

Table 5.4: Phase field discrete distance (PFDD) between different integration schemes using operator splitting corrections. Refer to Figure 5.9 for the experiment setup. Note that the distance between the schemes employing correction term modification (S-I corr.) and repeated time stepping (S-I 3 reps.) is about an order of magnitude smaller then between any two other schemes.

5.4 Computation time comparison

Lastly, we compare the simulation running times to the reference parallel implementation in [Str12].

Since our implementation is limited to one GPU we do not attempt to determine strong or weak scaling. Instead we only compare the absolute running times on a fixed hardware. Artificially lowering the number of GPU threads available to our implementation is possible but is not practical and does not extrapolate to multi-GPU setups.

$\Omega = (0, 4 \cdot n/512) \times (0, 4 \cdot n/512)$	$\mathcal{T} = [0, 0.04]$
$n_x = n_y = n$	$\Delta t = 5 \times 10^{-5}$
$\Delta x = \Delta y = 4/512$	$\xi = 0.0043$
$a = 2$	$\alpha = 3$
$b = 1$	$\beta = 1400$
$L = 2$	$T^* = 1$
$S = 0$	$m = 6$
$\theta_0 = 0$	

Table 5.5: Benchmark model parameters

The implementations were compared on simulation setup given in Table 5.5 for values of $n \in \{128, 256, 512, 1024, 2048\}$. Homogeneous Neumnann boundary conditions were used for both temperature and phase fields along with initial conditions from Section 5.1.2. The anisotropy strength S was set to 0 to be able to compare against the reference implementation (employing different model of anisotropy). The size of the computational domain Ω is scaled with n causing $\Delta x, \Delta y$ to remain constant. This ensures the stability criterion for explicit schemes remains unchanged for all values of n. Similar parameters were used to obtain Figure 5.4 with $S = 0$.

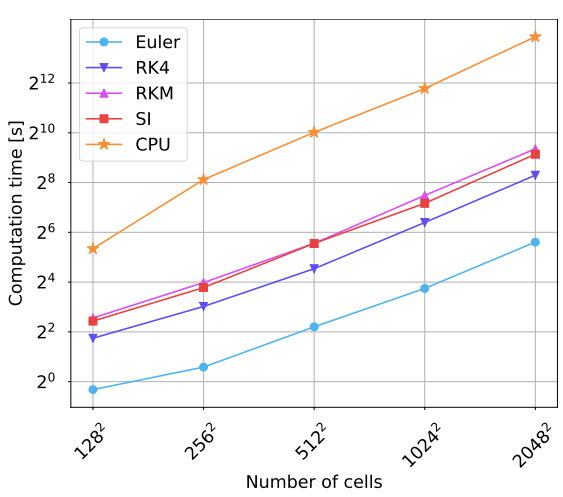
The implementation comparison was run on two different sets of hardware we will denote "consumer" and "HPC":

- Consumer: NVIDIA GeForce MX450 w. 1.8GB RAM, 8-core 11th Gen Intel(R) Core(TM) i5-1135G7 @ 2.40GHz (SMT mode enabled), 16 GB RAM

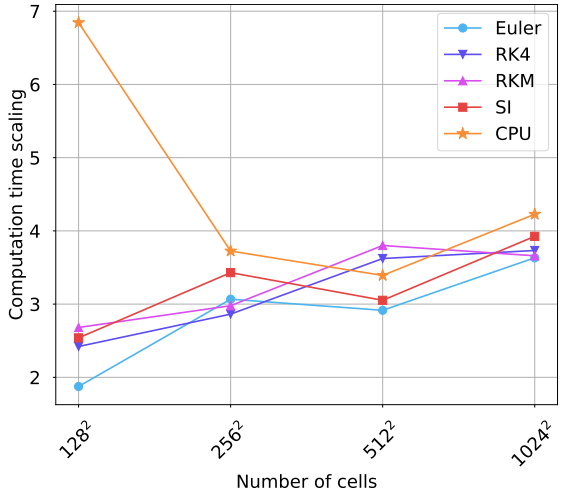
- HPC: NVIDIA A100 SXM4 w. 80GB HBM2 RAM, 32-core AMD EPYC 7543 @ 2.8GHz CPU (SMT mode disabled), 1024 GB RAM

Consumer hardware is lower end of current available CUDA enabled GPUs while HPC is hardware for compute clusters well suited for scientific computation. Most other GPUs are expected to lie somewhere between these two. The HPC hardware was accessible on the faculty supercomputer HELIOS.

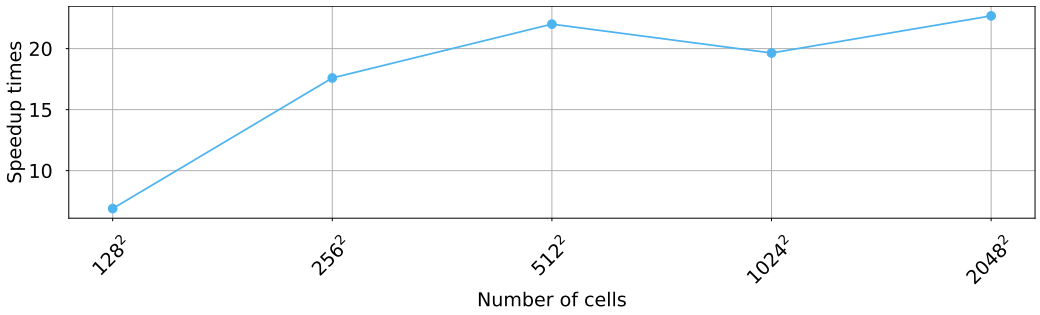
The results are summarized in Figures 5.11a, 5.12a. Note that the reference implementation employs the RKM integration scheme, and thus only that scheme should be used for direct comparison.



(a) Computation time



(b) Computation time scaling

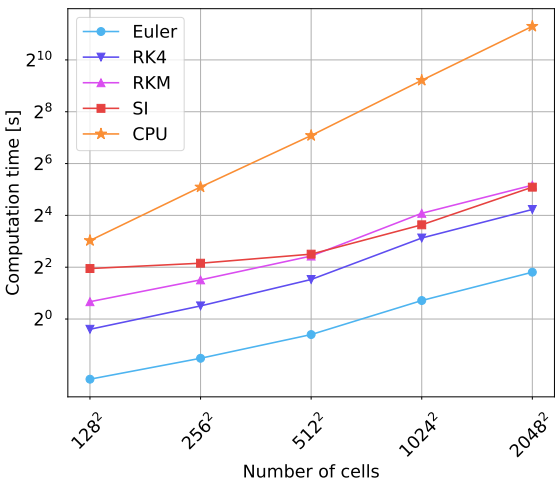


(c) RKM scheme speedup (reference / our implementation)

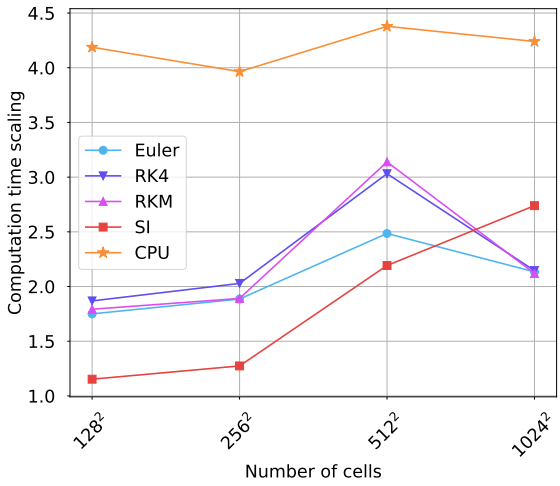
Figure 5.11: Computation time on consumer hardware

In Figure 5.11b, the scaling value $\text{Scale}(N)$ at $N \in \{256^2, 512^2, 1024^2, 2048^2\}$ is obtained by $\text{Scale}(N) = \text{Comp}(N)/\text{Comp}(N/4)$, where $\text{Comp}(N)$ is the computation time for N cells. Because the number of cells grows by a factor of 4 between data points and the simulation computation time is proportional to the number of cells, we would expect the scaling to be 4. However, we observe that for different integration schemes and implementations, the scalings differ. We explain the lower (better) then expected scaling for small sizes by the fact that that there is not enough computation to be done to fully utilize the entire GPU. Increasing the number

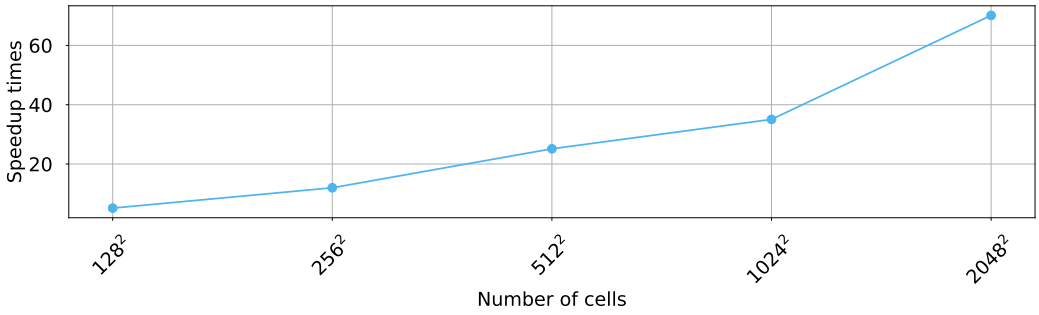
of cells N yields better utilization, resulting in lower scaling. This effect is more prevalent on the more capable HPC GPU hardware in Figure 5.12b.



(a) Computation time



(b) Computation time scaling



(c) RKM scheme speedup (reference / our implementation)

Figure 5.12: Computation time on HPC hardware

From Figure 5.11 we conclude that the goal of good simulation performance on consumer grade hardware was achieved. On the other hand, Figure 5.12 shows that the employed implementation scales well to more powerful GPU setups.

Conclusion

This work presents the derivation of several numerical schemes for solving the two dimensional phase field problem with a simple anisotropy, together with its GPU parallel implementation showing good performance on consumer hardware. The finite volume method notation is introduced and utilized to derive approximations for the Laplacian and gradient differential operators on admissible meshes. Different boundary conditions are described within the finite volume framework using ghost cells. Semi-discrete scheme of the phase field model is derived. Then explicit time integration schemes such as the explict Euler and Runge-Kutta-Merson methos are discussed and a time integration algorithm is presented. Semi-implicit time integration scheme utilizing the Crank-Nicolson method is derived. Solution of the resulting matrix of equations is discussed and operator splitting method is utilized to aid numerical matrix solvers and enabling the conjugate gradient method to be used. Internal error introduced by the operator splitting method is quantified and two techniques are provided for its reduction. The first is the repeated iteration technique, which has been shown to reliably reduce the operator splitting error. The second is the correction term technique, which has failed to reduce the operator splitting error, but produces similar crystal structures to the repeated iteration technique at no additional runtime cost.

Next, a detailed introduction to the CUDA hardware and programming model is given. Even though the text starts with simple examples, it quickly reaches non-trivial optimized implementations of the parallel for, parallel tiled for and parallel reduction algorithms. Special focus is put on shared memory with relation to the CUDA programming model and optimization of memory intensive kernels. A state-of-the-art parallel reduction kernel is presented, utilizing warp-level parallelism. Benchmarks of the presented algorithms are performed, showcasing superior performance on small and large datasets compared to the CUDA Thrust library.

Finally, simulation results of the proposed time integration schemes are shown. A discussion of boundary conditions and their impact on the simulation is given. Integration schemes are compared, showcasing consistency between the different techniques. The runtime performance of the developed simulation code is compared against a reference implementation. Speedups upwards of 20 times can be observed on both consumer hardware and specialized HPC hardware. The developed simulation code is freely available at https://github.com/Boostibot/bachelors.

Further work is needed to extend the proposed algorithms to three dimensions and efficiently distribute the simulation workload in many GPU setups, enabling simulation on high resolution three dimensional meshes. The parallel algorithms developed in this work can be used with advantage to solve more complex models including, for example, phase transitions in alloys, solidification subject to fluid flow, or freezing and thawing in porous media.

Bibliography

[AC79] Samuel M. Allen and John W. Cahn. "A microscopic theory for antiphase boundary motion and its application to antiphase domain coarsening." In: *Acta Metallurgica* 27.6 (1979), pp. 1085–1095. ISSN: 0001-6160. DOI: https://doi.org/10.1016/0001-6160(79)90196-2. URL: https://www.sciencedirect.com/science/article/pii/0001616079901962.

[AMW00] D.M. Anderson, G.B. McFadden, and A.A. Wheeler. "A phase-field model of solidification with convection." In: *Physica D: Nonlinear Phenomena* 135.1 (2000), pp. 175–194. ISSN: 0167-2789. DOI: https://doi.org/10.1016/S0167-2789(99)00109-8. URL: https://www.sciencedirect.com/science/article/pii/S0167278999001098.

[Bas+17] Steffen Basting et al. "Extended ALE Method for fluid–structure interaction problems with large structural displacements." In: *Journal of Computational Physics* 331 (2017), pp. 312–336. ISSN: 0021-9991. DOI: https://doi.org/10.1016/j.jcp.2016.11.043. URL: https://www.sciencedirect.com/science/article/pii/S0021999116306350.

[BDP22] Juan Bajo, Claudio Delrieux, and Gustavo Patow. "A Comprehensive Method for Liquid-to-Solid Interactions." In: *CLEI Electronic Journal* 25 (Apr. 2022). DOI: 10.19153/cleiej.25.1.4.

[Ben01a] M. Beneš. "Mathematical analysis of phase-field equations with numerically efficient coupling terms." In: *Interfaces and Free Boundaries - INTERFACE FREE BOUND* 3 (June 2001), pp. 201–212. DOI: 10.4171/IFB/38.

[Ben01b] M. Beneš. "Mathematical and computational aspects of solidification of pure substances." In: *Acta Math. Univ. Comenianae* (2001).

[Ben03] Michal Beneš. "Diffuse-interface treatment of the anisotropic mean-curvature flow." eng. In: *Applications of Mathematics* 48.6 (2003), pp. 437–453. URL: http://eudml.org/doc/33159.

[Ben97] M. Beneš. "Phase field model or microstructure growth in solidification of pure substances." PhD thesis. Czech Technical University in Prague, 1997.

[Bla15] Jiri Blazek. "Chapter 1 - Introduction." In: *Computational Fluid Dynamics: Principles and Applications (Third Edition)*. Ed. by Jiri Blazek. Third Edition. Oxford: Butterworth-Heinemann, 2015, pp. 1–5. ISBN: 978-0-08-099995-1. DOI: https://doi.org/10.1016/B978-0-08-099995-1.00001-4. URL: https://www.sciencedirect.com/science/article/pii/B9780080999951000014.

[BP96] G. Bellettini and M. Paolini. "Anisotropic motion by mean curvature in the context of Finsler geometry." In: *Hokkaido Mathematical Journal* 25.3 (1996), pp. 537–566. DOI: 10.14492/hokmj/1351516749. URL: https://doi.org/10.14492/hokmj/1351516749.

[Cag89] G. Caginalp. "Stefan and Hele-Shaw type models as asymptotic limits of the phase-field equations." In: *Phys. Rev. A* 39 (11 June 1989), pp. 5887–5896. DOI: 10.1103/PhysRevA.39.5887. URL: https://link.aps.org/doi/10.1103/PhysRevA.39.5887.

[Chr70] Jan Christiansen. "Numerical solution of ordinary simultaneous differential equations of the 1st order using a method for automatic step change." In: *Numerische Mathematik* 14.4 (1970), pp. 317–324. ISSN: 0945-3245. DOI: 10.1007/BF02165587. URL: https://doi.org/10.1007/BF02165587.

[CN47] J. Crank and P. Nicolson. "A practical method for numerical evaluation of solutions of partial differential equations of the heat-conduction type." In: *Mathematical Proceedings of the Cambridge Philosophical Society* 43.1 (1947), pp. 50–67. DOI: 10.1017/S0305004100023197.

[Cor24] NVIDIA Corporation. *CUDA C++ Programming Guide*. Version 12.5. 2024. URL: https://docs.nvidia.com/cuda/pdf/CUDA_C_Programming_Guide.pdf.

[Fow89] A. C. Fowler. "Secondary Frost Heave in Freezing Soils." In: *SIAM Journal on Applied Mathematics* 49.4 (1989), pp. 991–1008. DOI: 10.1137/0149060. eprint: https://doi.org/10.1137/0149060. URL: https://doi.org/10.1137/0149060.

[Gal+14] Stany Gallier et al. "A fictitious domain approach for the simulation of dense suspensions." In: *Journal of Computational Physics* 256 (2014), pp. 367–387. ISSN: 0021-9991. DOI: https://doi.org/10.1016/j.jcp.2013.09.015. URL: https://www.sciencedirect.com/science/article/pii/S0021999113006207.

[Gil80] R. R. Gilpin. "A model for the prediction of ice lensing and frost heave in soils." In: *Water Resources Research* 16.5 (1980), pp. 918–930. DOI: https://doi.org/10.1029/WR016i005p00918. eprint: https://agupubs.onlinelibrary.wiley.com/doi/pdf/10.1029/WR016i005p00918. URL: https://agupubs.onlinelibrary.wiley.com/doi/abs/10.1029/WR016i005p00918.

[Gur93] M. E. Gurtin. "Thermomechanics of evolving phase boundaries in the plane, Oxford Math-ematical Monographs." In: *Oxford University Press* (1993).

[Jes23] Taabish Jeshani. "Dynamically Finding Optimal Kernel Launch Parameters for CUDA Programs." MA thesis. Western University, 2023. URL: https://ir.lib.uwo.ca/etd/9268.

[JT96] A.A. Johnson and T.E. Tezduyar. "Simulation of multiple spheres falling in a liquid-filled tube." In: *Computer Methods in Applied Mechanics and Engineering* 134.3 (1996), pp. 351–373. ISSN: 0045-7825. DOI: https://doi.org/10.1016/0045-7825(95)00988-4. URL: https://www.sciencedirect.com/science/article/pii/0045782595009884.

[Kob93] Ryo Kobayashi. "Modeling and numerical simulations of dendritic crystal growth." In: *Physica D: Nonlinear Phenomena* 63.3 (1993), pp. 410–423. ISSN: 0167-2789. DOI: https://doi.org/10.1016/0167-2789(93)90120-P. URL: https://www.sciencedirect.com/science/article/pii/016727899390120P.

[LBK01] Yili Lu, C. Beckermann, and A. Karma. "Convection Effects in Three-Dimensional Dendritic Growth." In: *MRS Online Proceedings Library Archive* 701 (Jan. 2001). DOI: 10.1557/proc-701-t2.2.1.

[LeV02] Randall J. LeVeque. "Boundary Conditions and Ghost Cells." In: *Finite Volume Methods for Hyperbolic Problems*. Cambridge Texts in Applied Mathematics. Cambridge University Press, 2002, pp. 129–138.

[MMD16] F. Moukalled, L. Mangani, and M. Darwish. *The Finite Volume Method in Computational Fluid Dynamics: An Advanced Introduction with OpenFOAM® and Matlab*. Fluid Mechanics and Its Applications. Springer International Publishing, 2016. ISBN: 9783319348643. URL: https://books.google.cz/books?id=Gk5SswEACAAJ.

[MS16] Shev MacNamara and Gilbert Strang. "Operator Splitting." In: *Splitting Methods in Communication, Imaging, Science, and Engineering*. Ed. by Roland Glowinski, Stanley J. Osher, and Wotao Yin. Cham: Springer International Publishing, 2016, pp. 95–114. ISBN: 978-3-319-41589-5. DOI: 10.1007/978-3-319-41589-5_3. URL: https://doi.org/10.1007/978-3-319-41589-5_3.

[Pal23] J. Palán. "Modely fázového pole v materiálových vědách a jejich numerické řešení." MA thesis. FJFI ČVUT v Praze, 2023. URL: https://dspace.cvut.cz/handle/10467/108537.

[Pes72] Charles S Peskin. "Flow patterns around heart valves: A numerical method." In: *Journal of Computational Physics* 10.2 (1972), pp. 252–271. ISSN: 0021-9991. DOI: https://doi.org/10.1016/0021-9991(72)90065-4. URL: https://www.sciencedirect.com/science/article/pii/0021999172900654.

[Raa98] D. Raabe. *Computational materials science: The simulation of materials microstructures and properties*. Wiley-VCH, 1998.

[Red+21] Martin Reder et al. "Phase-field formulation of a fictitious domain method for particulate flows interacting with complex and evolving geometries." In: *International Journal for Numerical Methods in Fluids* 93 (Mar. 2021). DOI: 10.1002/fld.4984.

[Sal05] Jorge A. Vieyra Salas. "Phase-Field Simulation of Solidification with Moving Solids." MA thesis. Massachusetts Institute of Technology, 2005.

[SB16] Pavel Strachota and Michal Beneš. "A Hybrid Parallel Numerical Algorithm for Three-Dimensional Phase Field Modeling of Crystal Growth." In: *Proceedings of the Conference Algoritmy* (2016), pp. 23–32. URL: http://www.iam.fmph.uniba.sk/amuc/ojs/index.php/algoritmy/article/view/391.

[SB18] Pavel Strachota and Michal Beneš. "Error estimate of the finite volume scheme for the Allen–Cahn equation." In: *BIT Numerical Mathematics* 58.2 (2018), pp. 489–507. ISSN: 1572-9125. DOI: 10.1007/s10543-017-0687-4. URL: https://doi.org/10.1007/s10543-017-0687-4.

[Sch96] Alfred Schmidt. "Computation of Three Dimensional Dendrites with Finite Elements." In: *Journal of Computational Physics* 125.2 (1996), pp. 293–312. ISSN: 0021-9991. DOI: https://doi.org/10.1006/jcph.1996.0095. URL: https://www.sciencedirect.com/science/article/pii/S0021999196900959.

[She94] Jonathan R Shewchuk. *An Introduction to the Conjugate Gradient Method Without the Agonizing Pain*. Tech. rep. USA: Carnegie Mellon University, 1994. URL: https://www.cs.cmu.edu/~quake-papers/painless-conjugate-gradient.pdf.

[Str12] P. Strachota. "Analysis and Application of Numerical Methods for Solving Nonlinear Reaction-Diffusion Equations." PhD thesis. FJFI ČVUT v Praze, 2012.

[SWB21] P Strachota, A Wodecki, and M Beneš. "Focusing the latent heat release in 3D phase field simulations of dendritic crystal growth." In: *Modelling and Simulation in Materials Science and Engineering* 29.6 (July 2021), p. 065009. ISSN: 1361-651X. DOI: 10.1088/1361-651x/ac0f55. URL: http://dx.doi.org/10.1088/1361-651X/ac0f55.

[Tho98] J.W. Thomas. *Numerical Partial Differential Equations: Finite Difference Methods*. Texts in Applied Mathematics. Springer New York, 1998. ISBN: 9780387979991. URL: https://books.google.cz/books?id=op5COPwUfX8C.

[Ton+01] X. Tong et al. "Phase-field simulations of dendritic crystal growth in a forced flow." In: *Phys. Rev. E* 63 (6 2001), p. 061601. DOI: 10.1103/PhysRevE.63.061601. URL: https://link.aps.org/doi/10.1103/PhysRevE.63.061601.

[WBM92] A. A. Wheeler, W. J. Boettinger, and G. B. McFadden. "Phase-field model for isothermal phase transitions in binary alloys." In: *Phys. Rev. A* 45 (10 1992), pp. 7424–7439. DOI: 10.1103/PhysRevA.45.7424. URL: https://link.aps.org/doi/10.1103/PhysRevA.45.7424.

[WMS93] A.A. Wheeler, B.T. Murray, and R.J. Schaefer. "Computation of dendrites using a phase field model." In: *Physica D: Nonlinear Phenomena* 66.1 (1993), pp. 243–262. ISSN: 0167-2789. DOI: https://doi.org/10.1016/0167-2789(93)90242-S. URL: https://www.sciencedirect.com/science/article/pii/016727899390242S.

[You50] David M Young. "Iterative methods for solving partial difference equations of elliptical type (PDF)." PhD thesis. Harvard University, 1950. URL: https://web.ma.utexas.edu/CNA/DMY/david_young_thesis.pdf.

[ŽB18] Alexandr Žák and Michal Beneš. "Micro-scale model of thermomechanics in solidifying saturated porous media." In: *Acta Physica Polonica A* 134.3 (2018), pp. 678–682.

[ŽBI23] Alexandr Žák, Michal Beneš, and Tissa H. Illangasekare. "Pore-scale model of freezing inception in a porous medium." In: *Computer Methods in Applied Mechanics and Engineering* 414 (2023), p. 116166. ISSN: 0045-7825. DOI: https://doi.org/10.1016/j.cma.2023.116166. URL: https://www.sciencedirect.com/science/article/pii/S0045782523002906.

[Zhe+17] Tianyuan Zheng et al. "A Thermo-Hydro-Mechanical Finite Element Model of Freezing in Porous Media—Thermo-Mechanically Consistent Formulation and Application to Ground Source Heat Pumps." In: May 2017. URL: https://upcommons.upc.edu/bitstream/handle/2117/190800/Coupled-2017-92-A%20thermo-hydro-mechanical%20finite.pdf.

[ZM13] M.M. Zhou and G. Meschke. "A three-phase thermo-hydro-mechanical finite element model for freezing soils." In: *International Journal for Numerical and Analytical Methods in Geomechanics* 37.18 (2013), pp. 3173–3193. DOI: https://doi.org/10.1002/nag.2184. eprint: https://onlinelibrary.wiley.com/doi/pdf/10.1002/nag.2184. URL: https://onlinelibrary.wiley.com/doi/abs/10.1002/nag.2184.